

**TRƯỜNG ĐẠI HỌC CÔNG NGHỆ KỸ THUẬT
THÀNH PHỐ HỒ CHÍ MINH**

KHOA CÔNG NGHỆ THÔNG TIN



HỌC PHẦN: TRÍ TUỆ NHÂN TẠO CHO IoT

BÁO CÁO TIỂU LUẬN CUỐI KHÓA

**HỌC ĐA NHIỆM TRÊN CIFAR-10:
PHÂN LOẠI SIÊU LỚP KẾT HỢP
DỰ ĐOÁN GÓC XOAY ẢNH TỰ GIÁM SÁT**

Đánh giá trong điều kiện hạn chế nhân huấn luyện

Mã đề tài:	M8
Sinh viên thực hiện:	Đình Xuân Huy
Mã số sinh viên:	23110102
Phân nhóm:	Đa nhiệm
Dữ liệu:	CIFAR-10
Lớp học phần:	Trí tuệ nhân tạo cho IoT
Giảng viên hướng dẫn:	Hồ Nhật Minh
Năm học:	2026–2027

Mã nguồn: <https://github.com/dinhxuanhuy/iot-m8-multitask>

Thành phố Hồ Chí Minh, 2026

Tóm tắt

Báo cáo này trình bày một quy trình thực nghiệm có thể tái lập để đánh giá ảnh hưởng của một *nhiệm vụ phụ tự giám sát* đối với bài toán phân loại ảnh khi ngân sách nhân bị giới hạn. Cụ thể, CIFAR-10 được gộp thành hai siêu lớp do đề tài quy ước là *Animal* (6 lớp gốc) và *Vehicle* (4 lớp gốc); một encoder ResNet-18 điều chỉnh cho ảnh 32×32 được dùng chung cho hai đầu ra: phân loại siêu lớp có giám sát và dự đoán góc xoay $k \in \{0, 1, 2, 3\}$ được sinh tự động bằng `torch.rot90`. Nhân của nhiệm vụ phụ không cần nhãn ngữ nghĩa, nên nhánh xoay có thể khai thác thêm phần ảnh đã bị ẩn nhãn của tập huấn luyện.

Khối thực nghiệm chính gồm ba cấu hình (**E1** baseline chỉ học siêu lớp, **E2-L** thêm nhánh xoay nhưng chỉ dùng ảnh có nhãn, **E2** thêm nhánh xoay và dùng toàn bộ D_{train}), ba mức tỷ lệ nhãn (10 %, 20 %, 50 %) và ba lần lặp ghép cặp, tổng cộng 27 lượt huấn luyện, mỗi lượt 100 epoch với tổng thời gian huấn luyện 8,26 giờ. Chỉ số chính là Macro-F1 trên toàn bộ 10.000 ảnh test; Accuracy, precision/recall/F1 từng lớp và ma trận nhầm lẫn được báo cáo kèm theo.

Kết quả chính. Tại mức nhãn 10 %, Macro-F1 trên tập test đạt 0,9319 cho **E1** và 0,9470 cho **E2** (chênh lệch ghép cặp trung bình +1,51 điểm phần trăm); tại mức 50 % nhãn, **E1** đạt 0,9690 và **E2** đạt 0,9777 (chênh lệch +0,88 điểm phần trăm). Ở cả ba lần lặp, cấu hình **E2** tại mức 10 % nhãn đều cho Macro-F1 test cao hơn **E1** (chênh lệch trung bình +1,51 điểm phần trăm, độ lệch chuẩn 0,06).

Đóng góp có thể kiểm chứng. (i) Một phép chia benchmark được đồng bộ (seed 2026) với các tập nhãn lồng nhau $D_L^{(10\%)} \subset D_L^{(20\%)} \subset D_L^{(50\%)}$ và chỉ mục đầy đủ được lưu lại; (ii) một giao thức kiểm tra việc che nhãn ở mức chỉ mục, bảo đảm nhãn thật của D_U không đi vào loss, không dùng để chọn positive pair và không tham gia chọn checkpoint; (iii) một khối kết quả ghép cặp theo lần lặp với chênh lệch Δ_s và độ lệch chuẩn tường minh, kèm phân tích trường hợp không cải thiện; (iv) cài đặt đầy đủ bốn cấu hình mở rộng contrastive **E3–E6** (NT-Xent và SupCon) với siêu tham số λ_c được khóa *chỉ bằng validation* qua một quy trình hai bước tách bạch thông tin; (v) các phép đo chi phí huấn luyện và suy luận được tách riêng, với kết luận về IoT được giới hạn đúng theo thiết bị đã đo.

Kết quả phần mở rộng contrastive. Trong điều kiện đã khảo sát, cấu hình mở rộng **có** cải thiện so với **E1: E6** tại 10 % nhãn đạt chênh lệch trung bình +2,60 điểm phần trăm và dương ở cả 3/3 lần lặp. Tuy nhiên mức cải thiện này vẫn nằm trong cùng bậc độ lớn với độ lệch chuẩn giữa các lần lặp, nên chỉ được coi là bằng chứng thăm dò. Cần nhấn mạnh ba lần lặp chỉ cho bằng chứng ban đầu: không có tuyên bố ý nghĩa thống kê nào được đưa ra, và kết luận chỉ áp dụng cho kiến trúc, hai siêu lớp, các mức nhãn và giao thức đã khảo sát.

Thứ tự kết quả còn cho hai nhận xét có tính cấu trúc, đều nhất quán về dấu ở cả ba lần lặp: SupCon *nghiêng hơn* NT-Xent ở cả hai điều kiện (có và không có nhánh xoay), và nhánh xoay *bổ trợ* nhánh contrastive — thêm nhánh xoay lên trên một nhánh contrastive đã có vẫn cải thiện thêm, trong khi nhánh xoay chạy một mình thì không khác baseline.

Giới hạn. Kết luận chỉ áp dụng cho phép gộp hai siêu lớp, các tỷ lệ nhãn và giao thức đã khảo sát. Ba lần lặp chỉ cung cấp bằng chứng ban đầu về độ ổn định, không đủ để khẳng định ý nghĩa thống kê. Kết

quả đo độ trễ trên CPU của máy tính dùng để huấn luyện không phải là bằng chứng về khả năng triển khai trên vi điều khiển. Không có kết quả nào trong báo cáo này được suy ra từ số liệu minh họa.

Mục lục

Tóm tắt	1
1 Đặt vấn đề và mục tiêu	9
1.1 Đặt vấn đề	9
1.2 Mục tiêu và câu hỏi nghiên cứu	9
1.3 Phạm vi	10
2 Tập dữ liệu và tiền xử lý	10
2.1 Dữ liệu và ánh xạ nhãn	10
2.2 Chia dữ liệu và ngân sách nhãn	11
2.3 Tiền xử lý và sinh nhãn góc xoay	12
3 Phương pháp đề xuất	13
3.1 Kiến trúc học đa nhiệm	13
3.2 Hàm mất mát và cách sử dụng dữ liệu	15
3.3 Mở rộng Contrastive Learning và SupCon	15
3.4 Đối chứng bắt buộc	16
4 Thiết kế thực nghiệm và chỉ số	17
4.1 Môi trường thực nghiệm	17
4.2 Cấu hình huấn luyện	18
4.3 Quy trình và nguyên tắc so sánh	19
4.4 Chỉ số và cách báo cáo	20
4.5 Chi phí suy luận và giới hạn kết luận về IoT	20
5 Kết quả thực nghiệm	21
5.1 Khối thực nghiệm chính	21
5.2 Chênh lệch ghép cặp	23
5.3 Hành vi hội tụ	23
5.4 Chỉ số từng lớp và phân tích lỗi	23
5.5 Chẩn đoán nhánh phụ: Rotation Accuracy	26
5.6 Chi phí huấn luyện và suy luận	27
6 Kết quả phần mở rộng: Contrastive Learning và SupCon	28
6.1 Nguyên tắc khóa siêu tham số	28
6.2 Kết quả và đối chứng ghép cặp	30
6.3 Phân tích	30
6.4 Chi phí của phần mở rộng	31
6.5 Kết luận của phần mở rộng	32
7 Thảo luận	32
7.1 Trả lời ba câu hỏi đánh giá	32

7.2	Đọc kết quả theo hướng xu hướng ngân sách nhân	32
7.3	Các yếu tố gây nhiễu và mối đe dọa đến tính hợp lệ	32
7.4	Những gì đã được cường chế bằng mã nguồn, không chỉ bằng văn bản	33
7.5	Vì sao lợi ích của contrastive không tự động	34
7.6	Đọc kết quả mở rộng: chọn đúng mục tiêu phụ, rồi mới tăng dữ liệu	34
7.7	Hướng cải thiện	35
8	Liên hệ với Edge AI và IoT	36
8.1	Vì sao chi phí suy luận mới là chi phí quyết định trên thiết bị biên	36
8.2	Đối chiếu với ngân sách tài nguyên của thiết bị IoT	36
8.3	Giao thức đo độ trễ và giới hạn kết luận	36
8.4	Hàm ý thiết kế cho ứng dụng IoT	37
9	Kết luận	37
9.1	Kết luận chính	37
9.2	Kết luận về phần mở rộng contrastive	38
9.3	Tiêu chí hoàn thành đã đối chiếu	38
9.4	Giới hạn của kết luận	38
9.5	Hướng phát triển	39
A	Phụ lục	39
A.1	Kiểm tra kỹ thuật tự động	39
A.2	Gói mã nguồn bàn giao	39
A.3	Chi tiết từng lượt chạy	42
A.4	Kiểm thử tự động	43
A.5	Bảng số liệu dạng CSV	44
A.6	Đường học của toàn bộ lịch sử huấn luyện	44
A.7	Ghi nhận về các thiết lập được đề xuất	44
	Tài liệu tham khảo	45

Danh sách hình vẽ

1	Ảnh CIFAR-10 theo 10 lớp gốc (6 ảnh mỗi lớp, lấy ngẫu nhiên với seed 2026) và phép gộp hai siêu lớp của đề tài. Khung xanh lá = Animal (nhãn mới 0, gồm 6 lớp gốc); khung xanh dương = Vehicle (nhãn mới 1, gồm 4 lớp gốc). Số trong ngoặc là nhãn gốc của CIFAR-10.	11
2	Thành phần dữ liệu của phép chia benchmark. (a) Số ảnh mỗi lớp gốc trên ba tập; (b) tỷ lệ hai siêu lớp 60/40 trên ba tập; (c) ngân sách nhân: D_L so với D_U trên 45.000 ảnh của D_{train}	12

- 3 Ba loại view của cùng một ảnh (ảnh đầu tiên của D_L tại mức 10 % nhãn, seed 42): ảnh gốc; view của nhánh phân loại; hai view của nhánh contrastive; và bốn phép xoay $k = 0..3$ của nhánh phụ. Lưu ý nhánh xoay dùng ảnh *nguyên bản* 32×32 chứ không dùng ảnh đã cắt. 13
- 4 Luồng huấn luyện hai nhánh của khối chính. Hai nhánh nhận *các view khác nhau*: nhánh trên dùng augmentation ngẫu nhiên của ảnh chuẩn, nhánh dưới dùng ảnh đã xoay 90k độ. Encoder và các lớp BatchNorm được dùng chung, nên thống kê chạy của mạng phụ thuộc vào việc có nhánh xoay hay không. Các cấu hình mở rộng **E3–E6** thêm một nhánh thứ ba đi qua cùng encoder rồi qua Projection Head (mục 3.3); ba loại view tương ứng được minh họa ở Hình 3. 14
- 5 Macro-F1 trên D_{test} theo ngân sách nhãn cho ba cấu hình. Cột là trung bình qua ba lần lặp, thanh sai số là một độ lệch chuẩn mẫu. Đường chấm ngang ở 0,60 đánh dấu mức Accuracy của bộ phân loại luôn đoán “Animal”; mọi cấu hình đều vượt mức này rõ rệt. 22
- 6 Đường học tại mức 10 % nhãn, trung bình qua ba lần lặp. (a) Loss huấn luyện gồm cả hai nhánh; (b) loss phân loại trên validation; (c) Macro-F1 validation. Ở cấu hình **E1** nhánh phụ không tồn tại nên L_{rot} không được định nghĩa. 22
- 7 Chênh lệch ghép cặp Δ_s so với **E1** ở cùng mức nhãn, tính bằng điểm phần trăm Macro-F1 trên D_{test} . Mỗi điểm là một lần lặp; vạch ngang là trung bình của ba lần lặp. Giá trị dương nghĩa là cấu hình đa nhiệm tốt hơn baseline. 23
- 8 Ma trận nhầm lẫn trên D_{test} tại mức 10 % nhãn, gộp ba lần lặp (tổng 30.000 dự đoán cho mỗi cấu hình). Ô ghi số lượng tuyệt đối và tỷ lệ theo hàng. Ma trận cho thấy phần lớn sai số tập trung ở lớp Vehicle bị dự đoán thành Animal. 24
- 9 Các trường hợp dự đoán sai trên D_{test} ở mức 10 % nhãn, seed 42, cho ba cấu hình của khối đối chứng chính. Mỗi hàng lấy sáu ảnh, chọn *trái đều* từ ca *sai nhưng rất tự tin* tới ca *sát nước*, nên hình cho thấy cả lỗi hệ thống lẫn các ca ranh giới. Trên mỗi ô: chỉ mục ảnh trong tập test, nhãn thật → nhãn mô hình đoán, và độ tự tin của dự đoán sai đó. Số bên trái là *tổng số ảnh sai* của cấu hình, giảm từ 636 (**E1**) và 645 (**E2-L**) xuống 482 (**E2**). Ảnh giữ nguyên 32×32 px với nội suy nearest. Xu hướng lặp lại gồm hai nhóm: (a) phương tiện bị đoán thành Animal (ô tô cũ, tàu, thuyền) và (b) động vật nhỏ hoặc bị khuất trong bối cảnh có phương tiện (chim đậu trên dây điện hay lan can, thú cưng trên ghế) bị đoán thành Vehicle. Nhóm (b) đúng với loại sai số đã thấy ở ma trận nhầm lẫn. Nguồn của ba lượt này được nói rõ ở đoạn dưới. 25
- 10 Khảo sát λ_c **chỉ trên Macro-F1 validation** (40 epoch, mức 10 % nhãn, seed 42). Đường đứt dọc là giá trị được khóa. Tập test không được đánh giá trong bước này. 29
- 11 Macro-F1 trên D_{test} của các cấu hình *đã có kết quả*, trung bình qua các lần lặp đã xong, so với mức 0,60 của bộ phân loại luôn đoán Animal. Khối chính (**E1**, **E2-L**, **E2**) có kết quả ở cả ba mức nhãn; khối mở rộng contrastive chỉ được khảo sát ở mức 10 % nhãn (xem Bảng 21). Hình được sinh lại tự động bởi `scripts/make_extension_figures.py`, nên số cột phản ánh đúng số cấu hình đã chạy xong tại thời điểm sinh báo cáo. 30

- 12 Chênh lệch ghép cặp Δ_s so với **E1** của từng cấu hình mở rộng, mỗi điểm là một lần lặp. Đường ngang là trung bình các lần lặp đã xong. Giá trị dương nghĩa là cấu hình đó tốt hơn baseline ở cùng ngân sách nhân. Nếu chưa có lượt mở rộng nào thì hình chỉ hiển thị thông báo chưa có dữ liệu — đây là hình sinh tự động, không phải hình minh họa. 31

Danh sách bảng

- 1 Ba câu hỏi đánh giá và nội dung tương ứng. Câu C1 và C2 được trả lời bằng khối thực nghiệm chính (mục 5.1); câu C3 bằng các phép đo chi phí ở mục 5.6. 9
- 2 Ánh xạ từ 10 lớp gốc của CIFAR-10 sang hai siêu lớp của đề tài. Theo thứ tự nhãn gốc, $\text{Animal} = \{2, 3, 4, 5, 6, 7\}$ và $\text{Vehicle} = \{0, 1, 8, 9\}$ 10
- 3 Quy mô và thành phần của ba tập dữ liệu sau phép gộp siêu lớp. Tập test được giữ riêng hoàn toàn và chỉ mở ra ở bước đánh giá cuối cùng. 11
- 4 Số ảnh huấn luyện theo từng mức tỷ lệ nhãn. D_L là tập có nhãn, $D_U = D_{\text{train}} \setminus D_L$ là phần ẩn nhãn; mọi tỷ lệ được tính trên 45.000 ảnh của D_{train} , không phải trên toàn bộ 60.000 ảnh. 12
- 5 Chuỗi kích thước tensor của backbone ResNet-18 điều chỉnh cho ảnh 32×32 . Các giá trị này được kiểm tra tự động bằng `scripts/self_test.py` trên môi trường thực nghiệm. 14
- 6 Ba cấu hình chính. **E1** và **E2** là cặp đối chứng chính; **E2-L** tách riêng tác dụng của việc mở rộng nguồn ảnh cho nhánh phụ. 17
- 7 Bốn cấu hình mở rộng contrastive. λ_c dùng chung cho mọi cấu hình và được khóa bằng validation trước khi đánh giá test (mục 6.1). SupCon chỉ dùng D_L ; NT-Xent dùng D_{train} vì không cần nhãn siêu lớp. 17
- 8 Môi trường thực nghiệm được ghi lại tự động vào `results/environment.json` và `results/self_test.json`. Số luồng CPU được ghim bằng `torch.set_num_threads` trước các phép đo độ trễ. 18
- 9 Cấu hình huấn luyện dùng chung cho cả ba cấu hình. Ba cấu hình ghép cặp dùng cùng trọng số khởi tạo của encoder và Superclass Head. 18
- 10 Kết quả trên D_{test} : Macro-F1 và Accuracy (“Acc.”) là trung bình qua ba lần lặp. Δ là chênh lệch Macro-F1 trung bình so với **E1** ở cùng mức nhãn, tính bằng điểm phần trăm; giá trị dương nghĩa là cấu hình đa nhiệm tốt hơn baseline. Độ lệch chuẩn qua ba lần lặp được báo cáo ở Bảng 11. 21
- 11 Macro-F1 trên D_{test} dạng trung bình $\pm$ độ lệch chuẩn mẫu qua ba lần lặp. Ba lần lặp chỉ cung cấp bằng chứng ban đầu về độ ổn định, không đủ để kết luận về ý nghĩa thống kê. . 21
- 12 Hành vi hội tụ. “Khoảng cách” là Macro-F1 validation tại epoch tốt nhất trừ giá trị tại epoch cuối; giá trị dương nghĩa là mô hình bắt đầu suy giảm trên validation trước khi kết thúc 100 epoch. Vì không dùng Early Stopping, quá trình vẫn chạy tới epoch 100 như thiết kế. 23
- 13 Precision, recall và F1 từng siêu lớp tại mức 10 % nhãn (trung bình ba lần lặp). Vì tỷ lệ Animal/Vehicle là 60/40, Accuracy đơn lẻ không phản ánh đầy đủ chất lượng của từng lớp. 24

- 14 Precision, recall và F1 từng siêu lớp tại mức 50 % nhãn (trung bình ba lần lặp). Vì tỷ lệ Animal/Vehicle là 60/40, Accuracy đơn lẻ không phản ánh đầy đủ chất lượng của từng lớp. 24
- 15 Rotation Accuracy chẩn đoán trên D_{val} , đánh giá đủ bốn phép xoay cho mỗi ảnh. Đây chỉ là chỉ số chẩn đoán nhánh phụ và không thay thế bằng chứng về Macro-F1 của nhiệm vụ chính. 26
- 16 Thời gian huấn luyện trung bình cho một lượt chạy 100 epoch, trung bình qua BA lần lặp (độ lệch chuẩn xem Bảng 11). **E2-L** và **E2** xử lý thêm minibatch cho nhiệm vụ xoay ở mỗi bước cập nhật, nên chi phí học cao hơn **E1** một cách có hệ thống. 27
- 17 Chi phí tham số của mô hình tại cấu hình mục 3.1. Đây là đại lượng *xác định* (đếm trên kiến trúc), **không** phải trung bình qua nhiều lần chạy. Rotation Head chỉ tồn tại trong giai đoạn huấn luyện; khi suy luận chỉ cần encoder và Superclass Head. Dung lượng tệp được đo bằng cách lưu riêng trọng số suy luận, không gộp optimizer state. 27
- 18 Chi phí *dựng view* cho nhánh xoay, đo riêng trên CPU, không gồm forward/backward. Bảng đo theo GIẢ ĐỊNH nhánh phụ duyệt hết tập nguồn mỗi epoch; trong mã nguồn thì B_R được cắt về đúng b_L , nên số view *thực dùng* mỗi epoch bằng $|D_L|$ ở cả **E2-L** và **E2**. Vì vậy chi phí dựng view thực tế của hai cấu hình gần như bằng nhau, và tỷ lệ nêu trên là chi phí của kịch bản giả định. 27
- 19 Độ trễ suy luận với batch = 1, tensor $1 \times 3 \times 32 \times 32$, FP32, `model.eval()` và `torch.inference_mode()`. Mỗi điều kiện khởi động 50 lượt rồi đo 1.000 lượt, lặp ba phiên. Cách tổng hợp ba phiên KHÔNG giống nhau: *mean* và *p95* là trung bình cộng của ba phiên, còn *median* là trung vị của ba trung vị phiên (xem `src/m8/benchmark.py`). Phép đo chỉ tính thời gian tính toán của mô hình, không gồm đọc ảnh và tiền xử lý. . . . 28
- 20 Khảo sát λ_c bằng **Macro-F1 validation** với 40 epoch, mức 10 % nhãn, seed 42. Tập test **không** được đánh giá trong bước này. Quy tắc khóa được khai báo trước: chọn *một* λ_c dùng chung cho mọi cấu hình mở rộng sao cho trung bình Macro-F1 validation qua hai cấu hình khảo sát là lớn nhất; giá trị được chọn là 1,00 (trung bình 0,9461). 29
- 21 Kết quả trên D_{test} , trung bình qua ba lần lặp. Δ là chênh lệch Macro-F1 ghép cặp so với **E1** ở cùng mức nhãn, đơn vị điểm phần trăm. Hai nhóm được ngăn bằng khoảng trắng: **E1/E2-L/E2** là khối chính; **E3–E6** là khối mở rộng contrastive. 30
- 22 Chi phí của khối mở rộng. Số tham số suy luận **không** thay đổi so với khối chính vì cả Rotation Head lẫn Projection Head đều bị bỏ khi suy luận. “Lượt ảnh phụ” là tổng số lượt ảnh mà các nhánh phụ xử lý trong một epoch; đây là lý do định lượng khiến các cấu hình mở rộng chậm hơn **E1**. 31
- 23 Đối chiếu dung lượng trọng số của mô hình với ngân sách bộ nhớ của thiết bị biên. Các dòng lượng tử hóa là *ước lượng số học đơn giản* từ số tham số, chưa bao gồm chi phí hiệu chỉnh, activation và bộ nhớ runtime; chúng **không** phải là kết quả đã đo. Hai dòng cuối chỉ mang tính đối chiếu định cỡ, không phải suy luận về khả năng triển khai. . . . 36
- 24 Đối chiếu tiêu chí hoàn thành của đề cương với trạng thái thực tế của báo cáo. 38
- 25 Kết quả kiểm tra kỹ thuật tự động trên môi trường thực nghiệm. Script trả mã thoát khác 0 nếu bất kỳ kiểm tra nào không đạt, nên quy trình có thể được dùng trong CI. 39

26	Chi tiết từng lượt chạy trong khối thực nghiệm chính. “Epoch tốt” là epoch được chọn theo Macro-F1 validation.	42
----	--	----

1 Đặt vấn đề và mục tiêu

1.1 Đặt vấn đề

Bài toán được khảo sát là phân loại ảnh CIFAR-10 vào hai siêu lớp do đề tài quy ước: *Animal* và *Vehicle*. Mô hình dùng chung một bộ trích xuất đặc trưng ResNet-18 và đồng thời học hai nhiệm vụ: nhiệm vụ chính là phân loại siêu lớp có giám sát, nhiệm vụ phụ là dự đoán phép xoay đã được áp dụng lên ảnh. Nhân của nhiệm vụ phụ được sinh tự động từ chính phép biến đổi, nhờ đó không cần nhãn ngữ nghĩa của ảnh [1, 2].

Động cơ nghiên cứu không nằm ở việc đề xuất một kiến trúc mới, mà ở việc *đánh giá khả năng khai thác tín hiệu tự giám sát khi số ảnh có nhãn dùng cho huấn luyện bị giới hạn*. Nhiệm vụ dự đoán góc xoay được vận dụng từ hướng nghiên cứu RotNet; việc ghép ResNet-18 với hai đầu ra không được xem là đóng góp kiến trúc. Cần nhấn mạnh một điểm dễ gây nhầm lẫn: *góc xoay ở đây là góc biến đổi nhân tạo*, không phải góc tư thế thực của đối tượng trong ảnh [2].

Ở cấu hình có sử dụng thêm ảnh không nhãn, thiết lập thực chất là *học bán giám sát* với một nhiệm vụ phụ tự giám sát. Thuật ngữ “học đa nhiệm” mô tả giai đoạn huấn luyện; khi suy luận, hệ thống chỉ giữ nhiệm vụ phân loại Animal/Vehicle. Vì vậy lợi ích cần kiểm chứng là *chất lượng phân loại*, chứ không phải mặc nhiên là giảm chi phí suy luận so với một mô hình đơn nhiệm có cùng backbone.

Ba câu hỏi đánh giá được dùng xuyên suốt báo cáo:

Câu hỏi	Nội dung đánh giá
C1 — Hiệu quả phân loại	M8 có thay đổi Macro-F1 và Accuracy so với baseline chỉ học phân loại siêu lớp trong cùng ngân sách nhân hay không?
C2 — Vai trò của dữ liệu	Kết quả khác nhau thế nào khi nhánh rotation chỉ dùng ảnh thuộc tập có nhãn và khi được dùng toàn bộ tập train?
C3 — Chi phí thực hiện	Chi phí huấn luyện tăng bao nhiêu; mô hình một đầu ra khi suy luận có số tham số, dung lượng và độ trễ như thế nào trên máy đo xác định?

Bảng 1: Ba câu hỏi đánh giá và nội dung tương ứng. Câu C1 và C2 được trả lời bằng khối thực nghiệm chính (mục 5.1); câu C3 bằng các phép đo chi phí ở mục 5.6.

1.2 Mục tiêu và câu hỏi nghiên cứu

Mục tiêu tổng quát. Xây dựng và đánh giá một quy trình thực nghiệm có thể tái lập để xác định ảnh hưởng của nhiệm vụ dự đoán góc xoay đối với phân loại siêu lớp tại các mức 10 %, 20 % và 50 % nhãn huấn luyện.

Mục tiêu mở rộng. Đề cương giữ hướng Contrastive Learning/SupCon ở phần *mở rộng tùy chọn* (các cấu hình E3–E6). Báo cáo này triển khai đầy đủ hướng đó và báo cáo kết quả riêng tại mục 6, sau khi phần bắt buộc đã hoàn tất.

Giả thuyết cần kiểm chứng. Nhiệm vụ phụ *có thể* cải thiện kết quả, đặc biệt khi ít nhãn. Đây là một giả thuyết khoa học, không phải chỉ tiêu cam kết. Báo cáo phải trình bày đầy đủ cả trường hợp không cải thiện hoặc suy giảm; không có yêu cầu nào buộc kết quả của M8 phải cao hơn baseline.

Nguyên tắc báo cáo trung thực. Mọi kết quả trong báo cáo này — kể cả kết quả âm — đều được giữ nguyên như khi chạy thực nghiệm. Không có số liệu minh họa nào được đặt vào vị trí kết quả thực nghiệm, và không có lượt chạy nào bị loại bỏ sau khi đã xem kết quả test. Mục tiêu không được điều chỉnh sau khi biết kết quả.

1.3 Phạm vi

Giữ nguyên CIFAR-10, ResNet-18 điều chỉnh cho ảnh 32×32 và thời gian dự kiến bốn tuần. Phần bắt buộc tập trung vào baseline và M8; Contrastive Learning/SupCon là mở rộng tùy chọn, được triển khai và báo cáo ở mục 6 nhưng *không* thuộc tiêu chí hoàn thành bắt buộc. Việc đánh giá chi phí suy luận phục vụ thảo luận về Edge AI, *không thay thế* thử nghiệm trên thiết bị IoT thực tế và *không mặc nhiên* chứng minh khả năng triển khai trên vi điều khiển.

Những gì báo cáo này không khẳng định: (i) không so sánh trực tiếp Accuracy hai lớp với bảng xếp hạng CIFAR-10 mười lớp; (ii) không khẳng định đặc trưng học được tổng quát hơn hay thuật toán là mới; (iii) không khẳng định tối ưu năng lượng; (iv) không khẳng định đã xây dựng một hệ thống IoT hoàn chỉnh.

2 Tập dữ liệu và tiền xử lý

2.1 Dữ liệu và ánh xạ nhãn

CIFAR-10 gồm 60.000 ảnh RGB kích thước 32×32 : 50.000 ảnh train chính thức và 10.000 ảnh test chính thức; mỗi lớp gốc có 5.000 ảnh train và 1.000 ảnh test [5]. Hai siêu lớp dưới đây là *phép gộp của đề tài*, không phải bộ nhãn siêu lớp chính thức của CIFAR-10. Việc ghi rõ điều này là cần thiết để tránh so sánh sai với các bảng xếp hạng chuẩn.

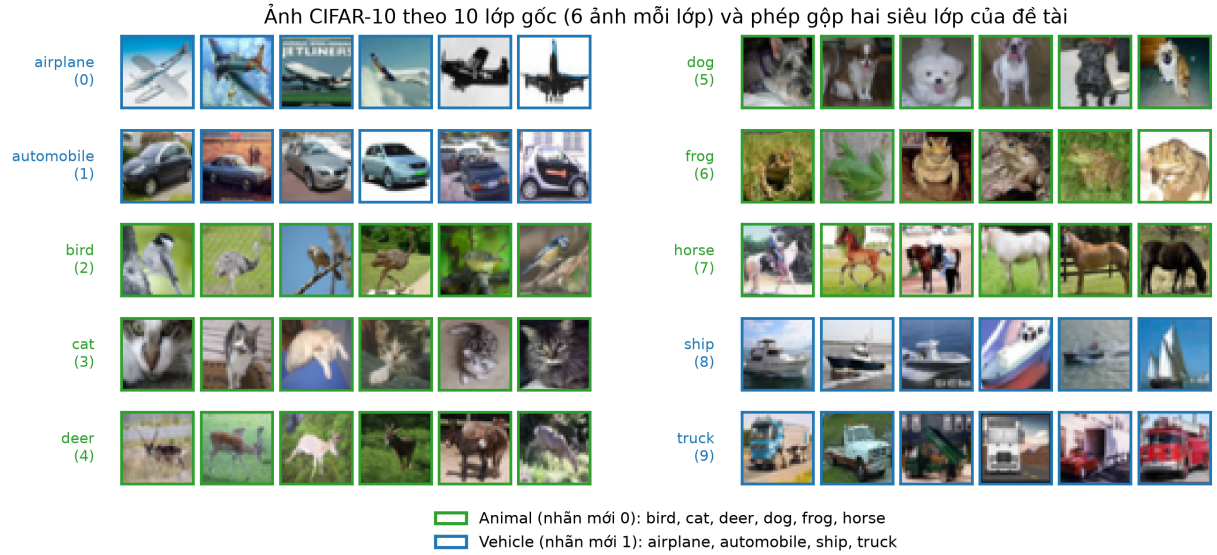
Nhãn mới	Siêu lớp	Các lớp gốc	Số lớp gốc
0	Animal	bird, cat, deer, dog, frog, horse	6
1	Vehicle	airplane, automobile, ship, truck	4

Bảng 2: Ánh xạ từ 10 lớp gốc của CIFAR-10 sang hai siêu lớp của đề tài. Theo thứ tự nhãn gốc, Animal = {2, 3, 4, 5, 6, 7} và Vehicle = {0, 1, 8, 9}.

Sau ánh xạ, tỷ lệ Animal/Vehicle là 60%/40% và không còn cân bằng. Hệ quả trực tiếp: một bộ phân loại luôn đoán “Animal” đã đạt Accuracy 60% trên phép chia này, nên Accuracy đơn lẻ không phản ánh chất lượng của lớp thiểu số. Do đó **Macro-F1 là chỉ số chính** và Accuracy chỉ là chỉ số bổ sung [23].

Hình 1 minh họa trực tiếp phép gộp này: mỗi hàng là một lớp gốc, màu khung thể hiện siêu lớp mà lớp đó thuộc về. Hình cũng cho thấy vì sao bài toán không tầm thường: cùng một siêu lớp chứa các đối tượng rất khác nhau (chim, mèo, hươu, chó, ếch, ngựa), trong khi hai siêu lớp lại có thể trông gần giống nhau ở kích thước ảnh 32×32 — ví dụ *airplane* nằm trên nền trời xanh có thể bị nhầm với *bird*.

Hình 2 trình bày phân bố lớp trên ba tập. Panel (a) xác nhận phép chia phân tầng giữ đúng 5.000 ảnh



Hình 1: Ảnh CIFAR-10 theo 10 lớp gốc (6 ảnh mỗi lớp, lấy ngẫu nhiên với seed 2026) và phép gộp hai siêu lớp của đề tài. Khung xanh lá = Animal (nhãn mới 0, gồm 6 lớp gốc); khung xanh dương = Vehicle (nhãn mới 1, gồm 4 lớp gốc). Số trong ngoặc là nhãn gốc của CIFAR-10.

Tập	Tổng số ảnh	Animal	Vehicle	Tỷ lệ Animal
D_{train}	45.000	27.000	18.000	60,0 %
D_{val}	5.000	3.000	2.000	60,0 %
D_{test}	10.000	6.000	4.000	60,0 %

Bảng 3: Quy mô và thành phần của ba tập dữ liệu sau phép gộp siêu lớp. Tập test được giữ riêng hoàn toàn và chỉ mở ra ở bước đánh giá cuối cùng.

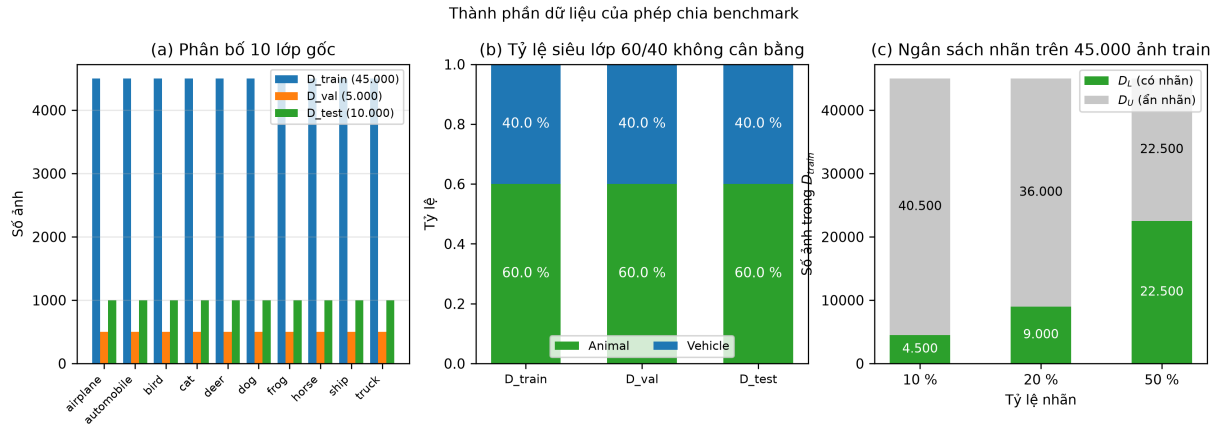
train và 1.000 ảnh test cho mỗi lớp gốc; panel (b) cho thấy tỷ lệ 60/40 được giữ nguyên trên cả D_{train} , D_{val} và D_{test} — điều này là cần thiết để chỉ số trên validation còn dự báo được cho test; panel (c) cho thấy ngân sách nhãn thay đổi thể nào theo ba mức tỷ lệ.

2.2 Chia dữ liệu và ngân sách nhãn

Quy trình chia dữ liệu được đóng băng trước khi huấn luyện và lưu đầy đủ chỉ mục:

- B1.** Dùng seed 2026 để chia 50.000 ảnh train thành D_{train} gồm 45.000 ảnh và D_{val} gồm 5.000 ảnh, *phân tầng theo 10 lớp gốc*. Kết quả phân tầng cho $D_{\text{val}} = 3.000 \text{ Animal} + 2.000 \text{ Vehicle}$.
- B2.** Tập test chính thức D_{test} được giữ riêng hoàn toàn, không tham gia bất kỳ bước nào của quá trình thiết kế và lựa chọn phương pháp.
- B3.** Trong mỗi lần lặp $s \in \{42, 43, 44\}$, tạo một thứ tự ngẫu nhiên theo từng lớp gốc trên D_{train} , rồi lấy các tập có nhãn lồng nhau $D_L^{(10\%)} \subset D_L^{(20\%)} \subset D_L^{(50\%)}$.
- B4.** $D_U = D_{\text{train}} \setminus D_L$ là phần ẩn nhãn; nhãn thật của D_U bị chặn ở mức chỉ mục và không bao giờ được đọc trong quá trình huấn luyện.

Một điểm dễ mắc lỗi cần được nêu rõ: mỗi mức tỷ lệ được tính trên **45.000 ảnh của D_{train}** , không phải trên toàn bộ 60.000 ảnh. Ngoài nhãn huấn luyện, 5.000 nhãn validation được dùng để chọn checkpoint;



Hình 2: Thành phần dữ liệu của phép chia benchmark. (a) Số ảnh mỗi lớp gốc trên ba tập; (b) tỷ lệ hai siêu lớp 60/40 trên ba tập; (c) ngân sách nhãn: D_L so với D_U trên 45.000 ảnh của D_{train} .

Tỷ lệ nhãn	D_L	Animal	Vehicle	D_U	Tổng D_{train}
10%	4.500	2.700	1.800	40.500	45.000
20%	9.000	5.400	3.600	36.000	45.000
50%	22.500	13.500	9.000	22.500	45.000

Bảng 4: Số ảnh huấn luyện theo từng mức tỷ lệ nhãn. D_L là tập có nhãn, $D_U = D_{\text{train}} \setminus D_L$ là phần ẩn nhãn; mọi tỷ lệ được tính trên 45.000 ảnh của D_{train} , không phải trên toàn bộ 60.000 ảnh.

đây là một khoản chi phí nhân thật và phải được khai báo riêng khi thảo luận về mức độ tiết kiệm nhãn. Tổng ngân sách nhãn thực tế của mỗi lượt chạy là $|D_L| + 5.000$.

Quy tắc chống rò rỉ nhãn (được kiểm tra tự động). Nhãn gốc chỉ phục vụ việc chuẩn bị phép chia benchmark. Nhãn thật của D_U *không* được đưa vào loss, *không* được dùng để chọn positive pair, *không* được dùng để huấn luyện bộ phân loại và *không* được dùng để lựa chọn mô hình. Script `scripts/prepare_data.py` kiểm tra và ghi lại: không trùng chỉ mục giữa train/validation/test, các tập D_L đúng số lượng của Bảng 4, các tập lồng nhau đúng thứ tự, và $D_L \cap D_U = \emptyset$. Kết quả kiểm tra đầy đủ được lưu trong `results/benchmark_audit.json`.

2.3 Tiền xử lý và sinh nhãn góc xoay

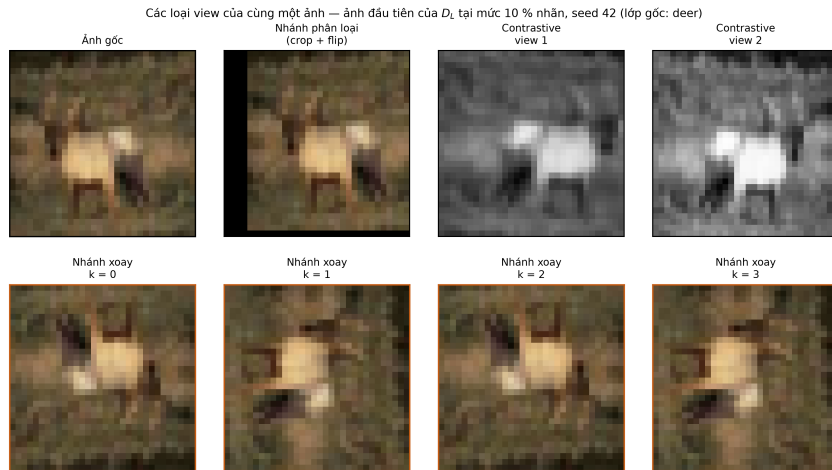
Hai nhánh sử dụng hai chuỗi biến đổi khác nhau, được mô tả tường minh để tránh hiểu nhầm rằng cả hai nhánh dùng chung một view.

Nhánh phân loại. `RandomCrop(32, padding=4) → RandomHorizontalFlip(p=0.5) → chuyển về tensor [0, 1] → Normalize`. Chuỗi này chỉ áp dụng khi huấn luyện.

Nhánh rotation. Dùng ảnh gốc 32×32 không cắt, chọn k đều trong $\{0, 1, 2, 3\}$, áp dụng $R_k = \text{xoay } 90k \text{ độ ngược chiều kim đồng hồ bằng } \text{torch.rot90}$, sau đó chuẩn hóa như trên [10]. **Không** thêm phép xoay hoặc lật nào sau bước gán k ; nếu thêm, nhãn k sẽ không còn khớp với biến đổi thực và nhiệm vụ phụ trở nên vô nghĩa.

Validation và test. Hai tập này chỉ chuyển tensor và chuẩn hóa, không augmentation ngẫu nhiên.

Hình 3 so sánh trực tiếp ba loại view của *cùng một ảnh*. Sự khác biệt là có chủ đích: nhánh phân loại cần một view giữ được thông tin ngữ nghĩa (cắt nhẹ, lật ngang), còn nhánh contrastive cần hai view “khó” của cùng một ảnh để việc kéo chúng lại gần nhau thực sự học được bất biến (cắt mạnh, đổi màu mạnh, có thể chuyển xám). Nhánh xoay thì **không** được cắt, vì cắt sẽ làm thay đổi góc xoay hiệu dụng và nhãn k không còn khớp với biến đổi thật.



Nhánh phân loại dùng RandomCrop + RandomHorizontalFlip; hai view contrastive dùng tăng cường mạnh độc lập (crop, flip, color jitter, grayscale); nhánh xoay dùng ảnh gốc 32×32 không cắt và xoay $90k$ độ ngược chiều kim đồng hồ.

Hình 3: Ba loại view của cùng một ảnh (ảnh đầu tiên của D_L tại mức 10 % nhãn, seed 42): ảnh gốc; view của nhánh phân loại; hai view của nhánh contrastive; và bốn phép xoay $k = 0..3$ của nhánh phụ. Lưu ý nhánh xoay dùng ảnh *nguyên bản* 32×32 chứ không dùng ảnh đã cắt.

Hằng số chuẩn hóa. $\text{mean} = \text{std} = (0,5; 0,5; 0,5)$. Đây là hằng số được chọn trước, áp dụng chung cho mọi cấu hình và mọi mức nhãn; giá trị này *không* được ước lượng từ validation hay test, vì làm như vậy sẽ tạo rò rỉ thông tin giữa các tập.

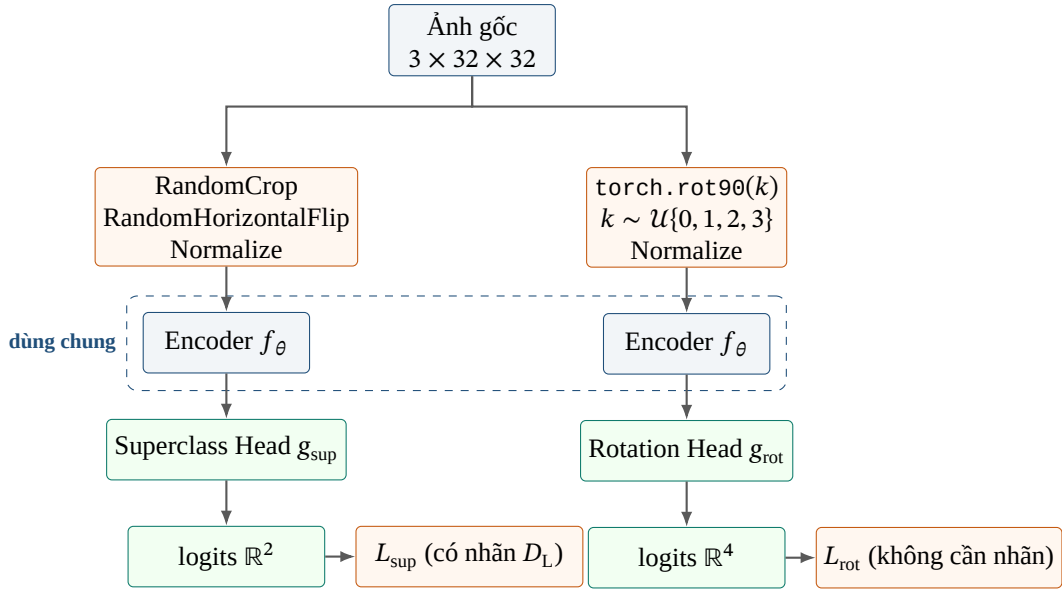
3 Phương pháp đề xuất

3.1 Kiến trúc học đa nhiệm

Backbone là ResNet-18 **khởi tạo ngẫu nhiên**, không dùng trọng số tiền huấn luyện. Việc không dùng tiền huấn luyện là lựa chọn có chủ đích: nếu encoder đã được huấn luyện trước trên ImageNet, hiệu ứng của nhiệm vụ phụ sẽ bị trộn lẫn với chất lượng đặc trưng sẵn có và không còn đo được đóng góp của tín hiệu tự giám sát [1, 6].

Điều chỉnh cho ảnh 32×32 :

- thay convolution đầu bằng `Conv2d(3, 64, kernel_size=3, stride=1, padding=1, bias=False)`;
- bỏ lớp MaxPool đầu mạng, vì sau hai lần giảm độ phân giải $32 \rightarrow 8$ thì thông tin không gian còn quá ít;



Hình 4: Luồng huấn luyện hai nhánh của khối chính. Hai nhánh nhận các view khác nhau: nhánh trên dùng augmentation ngẫu nhiên của ảnh chuẩn, nhánh dưới dùng ảnh đã xoay 90k độ. Encoder và các lớp BatchNorm được dùng chung, nên thống kê chạy của mạng phụ thuộc vào việc có nhánh xoay hay không. Các cấu hình mở rộng **E3–E6** thêm một nhánh thứ ba đi qua cùng encoder rồi qua Projection Head (mục 3.3); ba loại view tương ứng được minh họa ở Hình 3.

- giữ bốn nhóm residual block, mỗi nhóm hai BasicBlock (cấu hình ResNet-18 chuẩn).

Thành phần	Kích thước đầu ra $C \times H \times W$
Ảnh đầu vào	$3 \times 32 \times 32$
Conv 3×3 + BatchNorm + ReLU	$64 \times 32 \times 32$
Nhóm residual 1: 2 BasicBlock	$64 \times 32 \times 32$
Nhóm residual 2: 2 BasicBlock	$128 \times 16 \times 16$
Nhóm residual 3: 2 BasicBlock	$256 \times 8 \times 8$
Nhóm residual 4: 2 BasicBlock	$512 \times 4 \times 4$
Global Average Pooling + Flatten	vector 512 chiều
Superclass Head (tuyến tính, có bias)	$512 \rightarrow 2$
Rotation Head (tuyến tính, có bias)	$512 \rightarrow 4$

Bảng 5: Chuỗi kích thước tensor của backbone ResNet-18 điều chỉnh cho ảnh 32×32 . Các giá trị này được kiểm tra tự động bằng scripts/self_test.py trên môi trường thực nghiệm.

Ký hiệu. f_θ là encoder dùng chung; g_{sup} và g_{rot} là hai head tương ứng; $\mathcal{N}$ là bước chuyển tensor và chuẩn hóa; R_k là phép xoay 90k độ. Hai đầu ra được viết:

$$a_{\text{sup}} = g_{\text{sup}}(f_\theta(t_{\text{sup}}(x))) \in \mathbb{R}^2, \quad (1)$$

$$a_{\text{rot}} = g_{\text{rot}}(f_\theta(\mathcal{N}(R_k(x)))) \in \mathbb{R}^4, \quad (2)$$

trong đó $t_{\text{sup}}(\cdot)$ là chuỗi augmentation của nhánh phân loại. Việc viết tách t_{sup} và $\mathcal{N} \circ R_k$ nhấn mạnh rằng **hai nhánh nhận các view khác nhau**; ta không giả định tồn tại một vector đặc trưng duy nhất dùng cho cả ảnh chuẩn và ảnh đã xoay.

Cách ghép minibatch. Khi huấn luyện M8, hai minibatch được ghép theo chiều batch trước khi qua encoder: $[B_L; B_R] \in \mathbb{R}^{(b_L+b_R) \times 3 \times 32 \times 32}$, sau đó tách đặc trưng trở lại đúng head. **Các lớp BatchNorm dùng chung**, do đó mọi thay đổi về thành phần của batch đều làm thay đổi thống kê chạy của toàn mạng [13]. Đây là cơ sở cho một giới hạn quan trọng được nêu ở mục 4.3: không thể diễn giải so sánh **E2-L/E1** như một phép tách hoàn toàn riêng tác dụng của nhân góc xoay khỏi mọi thay đổi về mặt tính toán.

Suy luận. Khi suy luận, chỉ chạy encoder và Superclass Head; chọn lớp bằng arg max của hai logits. Rotation Head không cần thiết và không được tính vào chi phí suy luận. Hệ quả kiến trúc: mô hình suy luận của **E2** có *đúng cùng* kiến trúc với **E1**, nên không thể kỳ vọng giảm độ phức tạp kiến trúc chỉ nhờ bỏ head phụ.

3.2 Hàm mất mát và cách sử dụng dữ liệu

Với B_L là minibatch có nhãn, B_R là minibatch cho nhiệm vụ xoay, và b_L, b_R là số mẫu tương ứng, hai mất mát được lấy **trung bình riêng**:

$$L_{\text{sup}} = \frac{1}{b_L} \sum_{i=1}^{b_L} \text{CE}(a_{\text{sup},i}, y_i), \quad (3)$$

$$L_{\text{rot}} = \frac{1}{b_R} \sum_{j=1}^{b_R} \text{CE}(a_{\text{rot},j}, k_j), \quad (4)$$

$$L = L_{\text{sup}} + \lambda_{\text{rot}} L_{\text{rot}}. \quad (5)$$

CE là cross-entropy cho nhãn lớp nguyên. Trong PyTorch, logits được truyền trực tiếp vào crossEntropyLoss, **không** áp dụng Softmax trước loss; việc áp Softmax hai lần là một lỗi phổ biến làm hỏng gradient [7]. Trong mã nguồn, logits được truyền nguyên vẹn vào lớp này.

Phân chia nguồn dữ liệu cho từng mất mát. Chỉ ảnh thuộc D_L đóng góp vào L_{sup} . Ngược lại, B_R lấy từ D_L hoặc từ D_{train} tùy cấu hình (Bảng 6). Chốt $\lambda_{\text{rot}} = 0,5$ cho thí nghiệm chính; các giá trị khác chỉ thuộc khảo sát độ nhạy đã khai báo trước.

Vì sao lấy trung bình riêng từng mất mát? Nếu gộp hai tập mẫu rồi lấy trung bình chung, trọng số hiệu dụng của nhánh xoay sẽ phụ thuộc vào tỷ lệ b_R/b_L , và như vậy λ_{rot} mất ý nghĩa so sánh giữa các mức nhãn. Lấy trung bình riêng giữ cho λ_{rot} là một hệ số có thể diễn giải được và giữ phép so sánh độ nhạy λ_{rot} nhất quán.

3.3 Mở rộng Contrastive Learning và SupCon

Nội dung này là **hướng mở rộng** của đề tài: nó không phải điều kiện hoàn thành phần chính và được chạy *sau* khi khối đối chứng **E1/E2-L/E2** đã hoàn tất. Bốn cấu hình **E3–E6** được triển khai và báo cáo

tại mục 6.

Contrastive Learning tự giám sát (NT-Xent). Tạo hai view của cùng một ảnh và đưa qua encoder cùng Projection Head; hai view này tạo thành positive pair. Có thể dùng ảnh từ D_{train} vì **không** cần nhãn siêu lớp. Theo tham chiếu SimCLR [3], mọi thành phần đều được khóa trước: loss NT-Xent, chuẩn hóa vector L_2 , nhiệt độ τ_{NT} , phép tăng cường ảnh, kích thước batch và chiều Projection Head. Với $2N$ view trong một minibatch và $j(i)$ là view còn lại của cùng ảnh:

$$L_{\text{NT-Xent}} = \frac{1}{2N} \sum_{i=1}^{2N} -\log \frac{\exp(s_{i,j(i)}/\tau_{\text{NT}})}{\sum_{k \neq i} \exp(s_{i,k}/\tau_{\text{NT}})}, \quad s_{i,k} = \frac{z_i^\top z_k}{\|z_i\|_2 \|z_k\|_2}. \quad (6)$$

Supervised Contrastive Learning (SupCon). Xây dựng positive pair từ **các view có cùng nhãn siêu lớp đã được cấp cho huấn luyện**. Trong đề tài này, SupCon chỉ sử dụng D_L ; **không** đọc nhãn thật của D_U . SupCon là học có giám sát, không được gọi là một nhiệm vụ tự giám sát [4]. Với $A(i) = \{a \neq i : \tilde{y}_a = \tilde{y}_i\}$ là tập positive của view i :

$$L_{\text{SupCon}} = \sum_{i=1}^{2N} \frac{-1}{|A(i)|} \sum_{a \in A(i)} \log \frac{\exp(s_{i,a}/\tau_{\text{Sup}})}{\sum_{k \neq i} \exp(s_{i,k}/\tau_{\text{Sup}})}. \quad (7)$$

Hàm mục tiêu đầy đủ của bốn cấu hình mở rộng. Ký hiệu L_{con} là số hạng contrastive dùng ở cấu hình đang xét: $L_{\text{con}} = L_{\text{NT-Xent}}$ với **E3** và **E5**, $L_{\text{con}} = L_{\text{SupCon}}$ với **E4** và **E6**. Khi đó hàm mục tiêu là

$$L^{(\text{E3})} = L^{(\text{E4})} = L_{\text{sup}} + \lambda_c L_{\text{con}}, \quad (8)$$

$$L^{(\text{E5})} = L^{(\text{E6})} = L_{\text{sup}} + \lambda_{\text{rot}} L_{\text{rot}} + \lambda_c L_{\text{con}}, \quad (9)$$

trong đó (8) ứng với hai cấu hình *không* có nhánh xoay (**E3**, **E4**) và (9) ứng với hai cấu hình có đủ cả ba nhánh (**E5**, **E6**). Hệ số $\lambda_{\text{rot}} = 0,5$ giữ nguyên như khối chính; λ_c dùng chung cho cả bốn cấu hình và được khóa bằng Macro-F1 *validation* trước khi đánh giá test (mục 6.1). Bảng 7 ghi rõ nguồn ảnh của từng số hạng.

Kiến trúc Projection Head. Hai lớp tuyến tính $512 \rightarrow 512 \rightarrow \text{dim}_{\text{proj}}$ với BatchNorm và ReLU ở lớp ẩn (theo SupCon/SimCLR). Head này **chỉ** tồn tại khi huấn luyện; khi suy luận, mô hình chỉ gồm encoder và Superclass Head, nên *số tham số suy luận không thay đổi* so với **E1/E2** (xem Bảng 17 và Bảng 22).

3.4 Đối chứng bắt buộc

Ba cấu hình chính được thiết kế để tách bạch hai yếu tố thường bị trộn lẫn: *việc thêm nhiệm vụ phụ và việc mở rộng nguồn ảnh cho nhiệm vụ phụ*.

Ba so sánh ghép cặp được dùng để trả lời các câu hỏi đã nêu:

ID	Cấu hình	Ảnh cho phân loại	Ảnh cho nhiệm vụ xoay
E1	Baseline chỉ học siêu lớp	D_L	Không có
E2-L	Thêm nhánh xoay, cùng tập nguồn	D_L	Chỉ D_L
E2	Thêm nhánh xoay, mở rộng nguồn ảnh	D_L	$D_{\text{train}} = D_L \cup D_U$

Bảng 6: Ba cấu hình chính. **E1** và **E2** là cặp đối chứng chính; **E2-L** tách riêng tác dụng của việc mở rộng nguồn ảnh cho nhánh phụ.

ID	Hàm mục tiêu	Nguồn ảnh cho xoay	Nguồn ảnh cho loss bổ sung
E3	$L_{\text{sup}} + \lambda_c L_{\text{NT-Xent}}$	Không có nhánh xoay	D_{train} ; không dùng nhãn siêu lớp
E4	$L_{\text{sup}} + \lambda_c L_{\text{SupCon}}$	Không có nhánh xoay	Chỉ D_L có nhãn hợp lệ
E5	$L_{\text{sup}} + \lambda_{\text{rot}} L_{\text{rot}} + \lambda_c L_{\text{NT-Xent}}$	D_{train}	NT-Xent: D_{train} ; rotation: D_{train}
E6	$L_{\text{sup}} + \lambda_{\text{rot}} L_{\text{rot}} + \lambda_c L_{\text{SupCon}}$	D_{train}	SupCon: D_L ; rotation: D_{train}

Bảng 7: Bốn cấu hình mở rộng contrastive. λ_c dùng chung cho mọi cấu hình và được khóa bằng validation trước khi đánh giá test (mục 6.1). SupCon chỉ dùng D_L ; NT-Xent dùng D_{train} vì không cần nhãn siêu lớp.

- **E2-L so với E1:** tác động của việc bổ sung nhánh rotation khi *không* mở rộng tập ảnh nguồn. Đây là phép so sánh cô lập nhất về mặt dữ liệu, nhưng vẫn còn khác biệt về tính toán.
- **E2 so với E2-L:** ảnh hưởng của việc mở rộng nguồn ảnh cho nhánh phụ (chỉ D_L so với toàn bộ D_{train}). Hai cấu hình này có cùng hàm mục tiêu và cùng số bước cập nhật, chỉ khác nguồn ảnh của B_R .
- **E2 so với E1:** hiệu quả tổng thể của M8 trong cùng ngân sách nhân; đây là so sánh trả lời trực tiếp câu hỏi C1.

Các so sánh được ghép cặp theo lần lặp và tỷ lệ nhân, tức $\Delta_s = \text{Macro-F1}_s(\text{M8}) - \text{Macro-F1}_s(\text{E1})$ tính riêng cho từng s rồi mới tổng hợp.

Giới hạn về tính công bằng của phép so sánh. E2-L/E2 vẫn xử lý thêm view và làm thay đổi thống kê BatchNorm so với **E1**. Do đó **không** diễn giải chúng là phép tách hoàn toàn riêng tác dụng của “nhãn góc xoay” khỏi mọi thay đổi về tính toán. Tương tự, **không** khẳng định so sánh công bằng về FLOPs hoặc thời gian chỉ vì ba cấu hình dùng cùng số epoch: số lượt ảnh được xử lý ở mỗi bước cập nhật là khác nhau.

4 Thiết kế thực nghiệm và chỉ số

4.1 Môi trường thực nghiệm

Toàn bộ thí nghiệm được chạy trong một môi trường ảo Python tách biệt nhằm bảo đảm khả năng tái lập. Thông tin môi trường dưới đây được đọc trực tiếp từ chính phiên bản phần mềm đã dùng để huấn luyện và đo thời gian, không suy ra từ phiên bản của trang tài liệu.

Tạo môi trường.

```

1 python -m venv .env
2 .env\Scripts\python.exe -m pip install --upgrade pip
3 .env\Scripts\python.exe -m pip install --index-url https://download.pytorch.org/whl/cu128`

```

Thành phần	Giá trị thực tế
Python	3.12.10
PyTorch	2.11.0+cu128
TorchVision	0.26.0+cu128
Bản dựng CUDA	12.8
GPU	NVIDIA GeForce RTX 5070 Laptop GPU
VRAM	7,960 GiB
CPU	Intel64 Family 6 Model 197 Stepping 2, GenuineIntel
Số luồng CPU	16
RAM	63,4 GiB

Bảng 8: Môi trường thực nghiệm được ghi lại tự động vào `results/environment.json` và `results/self_test.json`. Số luồng CPU được ghim bằng `torch.set_num_threads` trước các phép đo độ trễ.

```
4 torch==2.11.0 torchvision==0.26.0
5 .venv\Scripts\python.exe -m pip install -r requirements.txt
```

Listing 1: Cài đặt môi trường ảo và các gói phụ thuộc.

4.2 Cấu hình huấn luyện

Để thống nhất số bước cập nhật có giám sát giữa các cấu hình, bảng kết quả chính **không dùng Early Stopping**. Cả ba cấu hình chạy đúng 100 epoch và đều được phép chọn checkpoint bằng cùng một tiêu chí validation. Early Stopping với `patience=10` của bản đề cương ban đầu chỉ có thể dùng cho các thử nghiệm thăm dò và phải được ghi rõ; nó **không** được trộn vào bảng so sánh chính vì sẽ tạo ra số epoch khác nhau giữa các cấu hình.

Thành phần	Thiết lập cho thí nghiệm chính
Optimizer	AdamW
Learning rate	3×10^{-4}
Weight decay	10^{-4}
Betas	(0,9; 0,999)
Eps	10^{-8}
Kích thước minibatch B_L	tối đa 128 ảnh
Kích thước minibatch B_R	bằng B_L tại mỗi bước
Ngân sách học	100 epoch (1 epoch = một lượt duyệt D_L)
Scheduler	CosineAnnealingLR, $T_{\max} = 100$, $\eta_{\min} = 0$
Hệ số λ_{rot}	0,5 cho thí nghiệm chính
Chọn checkpoint	Macro-F1 validation cao nhất; hòa thì loss thấp hơn, rồi epoch sớm hơn
Early stopping	Không dùng trong bảng kết quả chính
Seed	chia split 2026; lần lặp 42, 43, 44
Chuẩn hóa	mean = std = (0,5; 0,5; 0,5)

Bảng 9: Cấu hình huấn luyện dùng chung cho cả ba cấu hình. Ba cấu hình ghép cặp dùng cùng trọng số khởi tạo của encoder và Superclass Head.

Một epoch được định nghĩa thế nào. Một epoch là *một lượt duyệt toàn bộ D_L* , không phải một lượt duyệt D_{train} . Nhờ định nghĩa này, ba mức nhãn có số epoch giống nhau và số bước cập nhật có giám sát tỷ lệ với $|D_L|$. Minibatch cuối nhỏ hơn 128 được *giữ lại* chứ không bỏ mẫu, nên không có ảnh nào bị loại khỏi huấn luyện.

Chọn checkpoint. Tiêu chí là Macro-F1 validation cao nhất. Khi hai epoch có cùng Macro-F1 (trong sai số 10^{-12}), ưu tiên loss phân loại validation thấp hơn; nếu vẫn bằng nhau, chọn epoch sớm hơn. Tập test không tham gia vào bước này dưới bất kỳ hình thức nào.

4.3 Quy trình và nguyên tắc so sánh

Trong mỗi epoch, duyệt D_L một lần với cùng chuỗi minibatch có giám sát cho các cấu hình ghép cặp: cùng seed cho DataLoader, cùng thứ tự mẫu, cùng phép augmentation. Với mỗi bước của M8, lấy B_R ngẫu nhiên từ nguồn ảnh quy định tại Bảng 6; dùng một bộ sinh số ngẫu nhiên **độc lập** để việc lấy ảnh và góc xoay không làm thay đổi augmentation của nhánh phân loại. Sau đó tính tổng loss và thực hiện một bước optimizer.

Vì sao cần bộ sinh số ngẫu nhiên độc lập? Nếu nhánh phụ dùng chung luồng RNG với nhánh chính, việc thêm nhánh phụ sẽ *dịch chuyển* chuỗi số ngẫu nhiên của nhánh chính. Khi đó hai cấu hình ghép cặp sẽ thấy các phép augmentation khác nhau, và chênh lệch Δ_s trở thành nhiễu ngẫu nhiên thay vì tín hiệu cần đo.

E1 không chạy nhánh rotation, kể cả forward. Đặt $\lambda_{\text{rot}} = 0$ nhưng vẫn đưa ảnh xoay qua BatchNorm **không** được xem là baseline **E1**, vì thống kê chạy của mạng vẫn có thể bị thay đổi: các lớp BatchNorm cập nhật running mean và running variance ngay cả khi gradient của nhánh đó bị nhân với 0. Trong mã nguồn, **E1** được khởi tạo *không* có Rotation Head và vòng huấn luyện bỏ hoàn toàn nhánh phụ.

Không dùng nhãn của D_U . Với mọi cấu hình, nhãn của D_U không được dùng để phân tầng minibatch huấn luyện, không được dùng để xây dựng mục tiêu học, và không được dùng cho bất kỳ quyết định thiết kế nào.

Quy mô khối thực nghiệm. Khối chính gồm 3 cấu hình $\times$ 3 mức nhãn $\times$ 3 lần lặp = 27 lượt huấn luyện. Ba lần lặp thay đổi *cả* tập con có nhãn *lẫn* ngẫu nhiên huấn luyện (khởi tạo, thứ tự dữ liệu, augmentation), trong khi phép chia train/validation/test được cố định. Đây **không** phải cross-validation: cùng một tập validation được dùng cho mọi lần lặp, nên ba lần lặp chỉ đo độ ổn định trước hai nguồn ngẫu nhiên chứ không ước lượng được phương sai do chính phép chia dữ liệu gây ra.

Khảo sát tùy chọn. $\lambda_{\text{rot}} \in \{0,1; 0,25; 0,5; 1,0\}$ tại mức 10 % nhãn, được ưu tiên trước các mở rộng E3–E6. Giữ $\lambda_{\text{rot}} = 0,5$ cho so sánh chính. Không được lựa chọn λ hay cấu hình dựa trên kết quả test. Nếu tài nguyên không đủ, phải điều chỉnh ngân sách chung và ghi nhận *trước khi* mở kết quả test.

Quy trình khóa siêu tham số của phần mở rộng. Khối mở rộng contrastive **E3–E6** có thêm ba siêu tham số phải khóa trước khi đánh giá test: nhiệt độ SupCon (0,1), nhiệt độ NT-Xent (0,2) và hệ số λ_c . Hai nhiệt độ và chiều Projection Head (128) được chốt theo giá trị chuẩn của tài liệu gốc [3, 4]; riêng λ_c được chọn bằng **Macro-F1 validation** qua một script *không* có bước đánh giá test (mục 6.1). Quy tắc chọn được khai báo trước khi chạy để tránh điều chỉnh giao thức sau khi đã thấy kết quả.

4.4 Chỉ số và cách báo cáo

Chỉ số chính là **Macro-F1** của Animal và Vehicle. Báo cáo thêm Accuracy, precision/recall/F1 từng lớp và ma trận nhầm lẫn trên toàn bộ 10.000 ảnh test. Mỗi ảnh chỉ nhận một dự đoán của nhánh siêu lớp; **không** dùng test-time augmentation trong thí nghiệm chính.

Các chỉ số được định nghĩa như sau:

$$\text{Accuracy} = \frac{1}{N} \sum_{i=1}^N \mathbb{1}[\hat{y}_i = y_i], \quad (10)$$

$$P_c = \frac{TP_c}{TP_c + FP_c}, \quad R_c = \frac{TP_c}{TP_c + FN_c}, \quad (11)$$

$$F1_c = \frac{2P_cR_c}{P_c + R_c}, \quad \text{Macro-F1} = \frac{F1_{\text{Animal}} + F1_{\text{Vehicle}}}{2}. \quad (12)$$

Quy ước biên. Nếu mẫu số của precision, recall hoặc F1 bằng 0, giá trị tương ứng được quy ước bằng 0 và dùng thống nhất cho mọi cấu hình. Chỉ số được tính từ *toàn bộ* dự đoán của tập đánh giá; không lấy trung bình không trọng số các F1 của từng minibatch, vì làm như vậy sẽ cho trọng số sai cho các minibatch nhỏ ở cuối epoch.

Cách tổng hợp. Báo cáo trung bình $\pm$ độ lệch chuẩn mẫu qua ba lần lặp và chênh lệch ghép cặp $\Delta_s = \text{Macro-F1}_s(\text{M8}) - \text{Macro-F1}_s(\mathbf{E1})$ tính bằng điểm phần trăm. Mức 10 % là so sánh ưu tiên; mức 20 % và 50 % dùng để đánh giá xu hướng.

Giới hạn suy luận thống kê. Ba lần lặp chỉ cho bằng chứng ban đầu về độ ổn định. Với $n = 3$ và độ lệch chuẩn quan sát được, khoảng tin cậy rất rộng và không có cơ sở để tuyên bố “có ý nghĩa thống kê” hay “luôn tốt hơn”. Đặc biệt, **không** được chọn một lần chạy tốt nhất rồi kết luận từ nó: báo cáo dùng trung bình và độ lệch chuẩn qua cả ba lần [24].

Rotation Accuracy. Chỉ là chỉ số *chẩn đoán* cho nhánh phụ. Nếu đo trên validation, phải đánh giá đủ bốn phép xoay cho mỗi ảnh và ghi rõ giao thức. Độ chính xác cao ở nhiệm vụ phụ **không** thay thế bằng chứng cải thiện Macro-F1 của nhiệm vụ chính; một nhiệm vụ phụ được học tốt vẫn hoàn toàn có thể không giúp gì cho nhiệm vụ chính.

4.5 Chi phí suy luận và giới hạn kết luận về IoT

Đếm tham số và dung lượng. Đếm riêng số tham số khi huấn luyện và khi suy luận; đo dung lượng tệp chỉ chứa trọng số suy luận, **không** gộp optimizer state (vì optimizer state không tồn tại trên thiết bị triển khai). Với cấu hình tại mục 3.1, mô hình suy luận có 11.169.858 tham số; Rotation Head thêm 2.052 tham số. Phần tham số FP32 của mô hình suy luận tương đương khoảng 42,61 MiB, *chưa* gồm buffer, activation và bộ nhớ của runtime [6]. Đây là thông số kiến trúc, không phải kết quả huấn luyện.

Giao thức đo độ trễ. Đo với $\text{batch} = 1$, tensor $1 \times 3 \times 32 \times 32$, FP32, `model.eval()` và `torch.inference_mode()`. Dùng cùng máy, cùng runtime và cùng số luồng CPU cho mọi cấu hình. Chạy khởi động 50 lượt, sau đó đo tối thiểu 1.000 lượt trên cùng các ảnh validation đã tiền xử lý. Nếu đo trên GPU, đồng bộ trước và sau khối tính thời gian. Báo cáo mean, median, p95 theo ms/ảnh; lặp ba phiên và lưu log [8].

Tách riêng các thành phần thời gian. Phép đo trên chỉ tính thời gian *suy luận của mô hình*. Thời gian đọc ảnh, tiền xử lý và truyền dữ liệu phải được đo và báo cáo riêng. Hai mô hình **E1** và **E2** có cùng kiến trúc suy luận, nên không kỳ vọng giảm độ phức tạp kiến trúc chỉ nhờ bỏ head phụ.

5 Kết quả thực nghiệm

Các *chỉ số kết quả* trong phần này được sinh tự động từ `results/checkpoints/*.json` (khối chính) và `results/checkpoints_ext/*.json` (khối mở rộng) do quá trình huấn luyện sinh ra, rồi tổng hợp bằng `scripts/make_report_data.py`. Không có chỉ số nào được nhập bằng tay.

Cần phân biệt rõ hai loại giá trị: (i) *chỉ số kết quả* — Macro-F1, Accuracy, độ lệch chuẩn, chênh lệch ghép cặp, thời gian huấn luyện, số tham số, độ trễ — đều đọc từ JSON/CSV do lần chạy thật sinh ra; (ii) *hằng số giao thức và kiến trúc* — phép chia dữ liệu, số tham số từng tầng, siêu tham số optimizer — được viết trong `make_report_data.py` theo `src/m8/config.py`. Các hằng số ở nhóm (ii) hiện khớp `config.py` và `results/self_test.json`, nhưng vì không đọc lại từ JSON nên nếu `config.py` đổi thì bảng sẽ không tự phát hiện.

5.1 Khối thực nghiệm chính

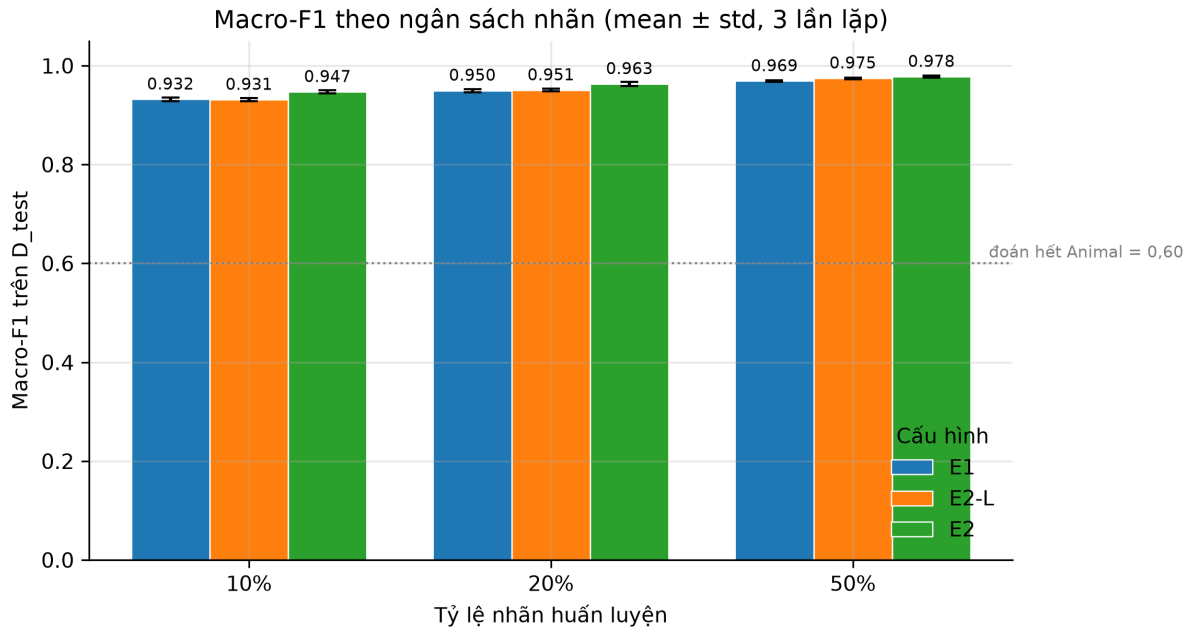
Khối thực nghiệm hoàn thành 27 lượt huấn luyện ($3 \text{ cấu hình} \times 3 \text{ mức nhãn} \times 3 \text{ lần lặp}$), tổng thời gian huấn luyện 8,26 giờ trên NVIDIA GeForce RTX 5070 Laptop GPU. Chi tiết từng lượt được liệt kê ở Bảng 26; bảng tổng hợp kết quả trên tập test là Bảng 10.

Cấu hình	10 % nhãn			20 % nhãn			50 % nhãn		
	Macro-F1	Acc.	Δ	Macro-F1	Acc.	Δ	Macro-F1	Acc.	Δ
E1	0,9319	0,9349	—	0,9497	0,9517	—	0,9690	0,9702	—
E2-L	0,9313	0,9340	−0,06	0,9505	0,9524	+0,08	0,9745	0,9755	+0,55
E2	0,9470	0,9492	+1,51	0,9631	0,9645	+1,34	0,9777	0,9786	+0,88

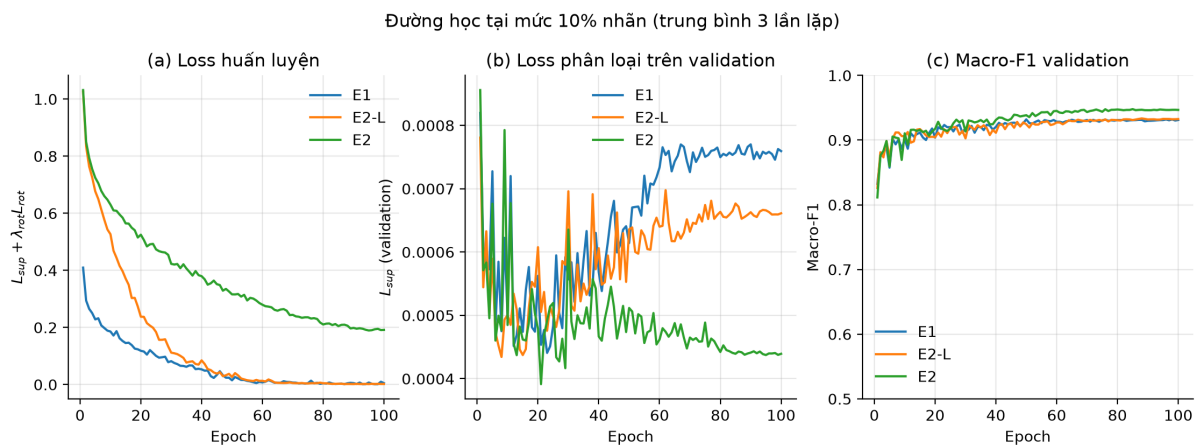
Bảng 10: Kết quả trên D_{test} : Macro-F1 và Accuracy (“Acc.”) là trung bình qua ba lần lặp. Δ là chênh lệch Macro-F1 trung bình so với **E1** ở cùng mức nhãn, tính bằng điểm phần trăm; giá trị dương nghĩa là cấu hình đa nhiệm tốt hơn baseline. Độ lệch chuẩn qua ba lần lặp được báo cáo ở Bảng 11.

Cấu hình	10 % nhãn	20 % nhãn	50 % nhãn
E1	$0,9319 \pm 0,0035$	$0,9497 \pm 0,0030$	$0,9690 \pm 0,0015$
E2-L	$0,9313 \pm 0,0030$	$0,9505 \pm 0,0027$	$0,9745 \pm 0,0017$
E2	$0,9470 \pm 0,0034$	$0,9631 \pm 0,0047$	$0,9777 \pm 0,0022$

Bảng 11: Macro-F1 trên D_{test} dạng trung bình $\pm$ độ lệch chuẩn mẫu qua ba lần lặp. Ba lần lặp chỉ cung cấp bằng chứng ban đầu về độ ổn định, không đủ để kết luận về ý nghĩa thống kê.



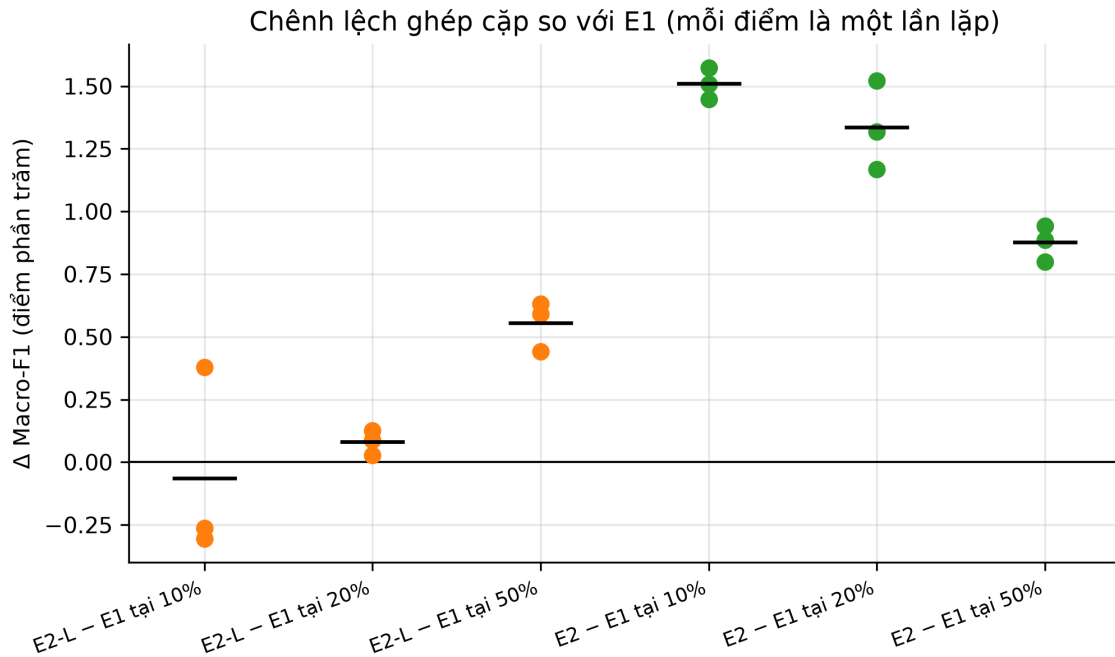
Hình 5: Macro-F1 trên D_{test} theo ngân sách nhãn cho ba cấu hình. Cột là trung bình qua ba lần lặp, thanh sai số là một độ lệch chuẩn mẫu. Đường chấm ngang ở 0,60 đánh dấu mức Accuracy của bộ phân loại luôn đoán “Animal”; mọi cấu hình đều vượt mức này rõ rệt.



Hình 6: Đường học tại mức 10 % nhãn, trung bình qua ba lần lặp. (a) Loss huấn luyện gồm cả hai nhánh; (b) loss phân loại trên validation; (c) Macro-F1 validation. Ở cấu hình **E1** nhánh phụ không tồn tại nên L_{rot} không được định nghĩa.

5.2 Chênh lệch ghép cặp

Vì ba lần lặp tương ứng với ba tập con nhãn khác nhau, phép so sánh có ý nghĩa nhất là so sánh *ghép cặp*: tính Δ_s cho từng lần lặp rồi mới tổng hợp, thay vì so sánh hai trung bình độc lập. Bảng 10 đã ghi giá trị trung bình của Δ ; hình 7 cho thấy phân bố của Δ_s theo từng lần lặp, nhờ đó có thể thấy mức độ nhất quán của hiệu ứng.



Hình 7: Chênh lệch ghép cặp Δ_s so với **E1** ở cùng mức nhãn, tính bằng điểm phần trăm Macro-F1 trên D_{test} . Mỗi điểm là một lần lặp; vạch ngang là trung bình của ba lần lặp. Giá trị dương nghĩa là cấu hình đa nhiệm tốt hơn baseline.

5.3 Hành vi hội tụ

Vì bảng kết quả chính không dùng Early Stopping, cần kiểm tra xem việc chạy đủ 100 epoch có dẫn tới quá khớp lâu dài trên validation hay không. Bảng 12 trình bày thời điểm đạt checkpoint tốt nhất và mức suy giảm về cuối quá trình.

Cấu hình	Epoch tốt nhất	Macro-F1 val tại epoch tốt nhất	Macro-F1 val tại epoch cuối	Khoảng cách
E1	90,8	0,9521	0,9507	+0,0014
E2-L	85,3	0,9542	0,9524	+0,0018
E2	85,0	0,9644	0,9631	+0,0014

Bảng 12: Hành vi hội tụ. “Khoảng cách” là Macro-F1 validation tại epoch tốt nhất trừ giá trị tại epoch cuối; giá trị dương nghĩa là mô hình bắt đầu suy giảm trên validation trước khi kết thúc 100 epoch. Vì không dùng Early Stopping, quá trình vẫn chạy tới epoch 100 như thiết kế.

5.4 Chỉ số từng lớp và phân tích lỗi

Bảng 13 và Bảng 14 trình bày precision, recall và F1 của từng siêu lớp tại hai mức nhãn. Điểm đáng chú ý: *Vehicle* luôn yếu hơn *Animal* ở mọi cấu hình và mọi mức nhãn, và **E2** cải thiện *Vehicle* nhiều

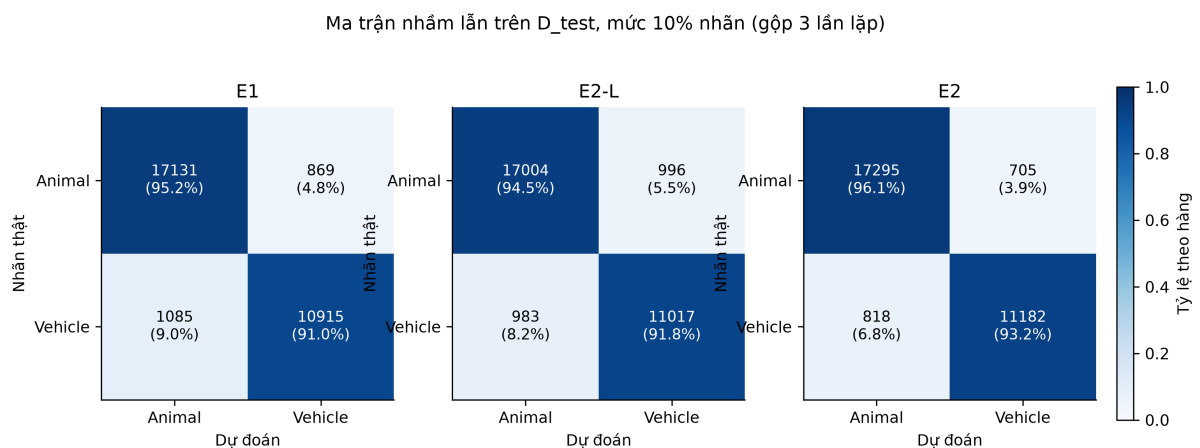
hơn Animal (+1,84 so với +1,17 điểm phần trăm tại 10 % nhân). Vì lợi ích tập trung ở lớp thiểu số nên Macro-F1 tăng nhiều hơn Accuracy — đúng lý do chọn Macro-F1 làm chỉ số chính.

Cấu hình	Animal		Vehicle		F1 (Animal / Vehicle)
	Precision	Recall	Precision	Recall	
E1	0,9405	0,9517	0,9263	0,9096	0,9461 / 0,9178
E2-L	0,9454	0,9447	0,9172	0,9181	0,9450 / 0,9176
E2	0,9548	0,9608	0,9407	0,9318	0,9578 / 0,9362

Bảng 13: Precision, recall và F1 từng siêu lớp tại mức 10 % nhân (trung bình ba lần lặp). Vì tỷ lệ Animal/Vehicle là 60/40, Accuracy đơn lẻ không phản ánh đầy đủ chất lượng của từng lớp.

Cấu hình	Animal		Vehicle		F1 (Animal / Vehicle)
	Precision	Recall	Precision	Recall	
E1	0,9735	0,9769	0,9652	0,9602	0,9752 / 0,9627
E2-L	0,9819	0,9772	0,9660	0,9730	0,9795 / 0,9695
E2	0,9838	0,9805	0,9709	0,9758	0,9821 / 0,9733

Bảng 14: Precision, recall và F1 từng siêu lớp tại mức 50 % nhân (trung bình ba lần lặp). Vì tỷ lệ Animal/Vehicle là 60/40, Accuracy đơn lẻ không phản ánh đầy đủ chất lượng của từng lớp.

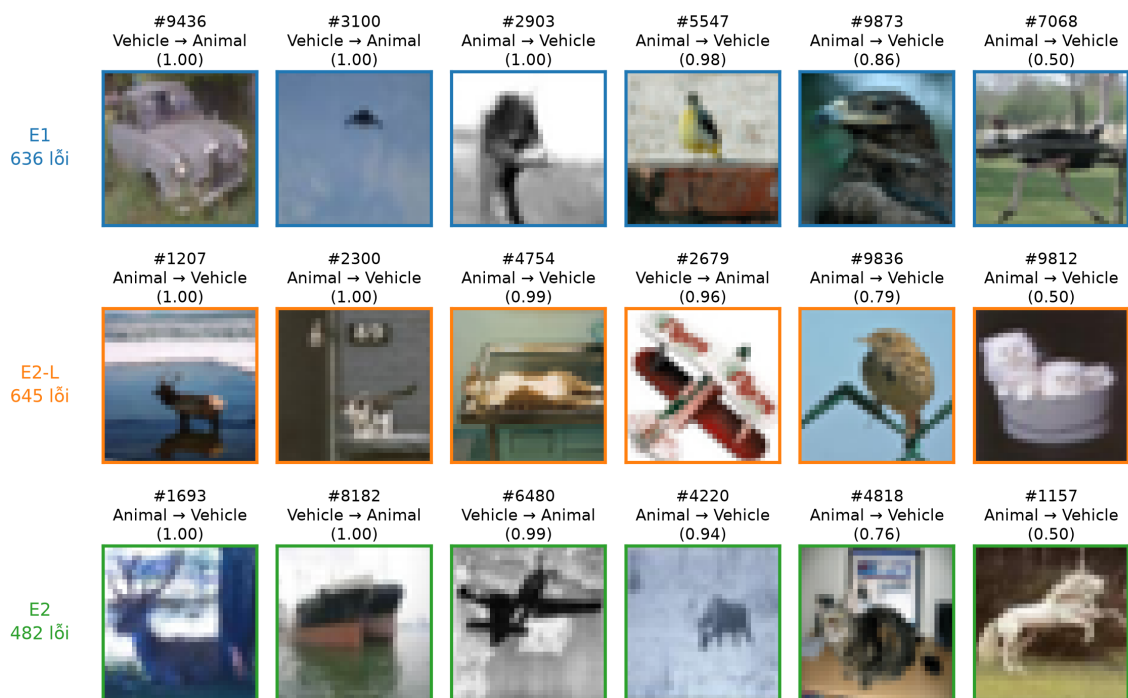


Hình 8: Ma trận nhầm lẫn trên D_{test} tại mức 10 % nhân, gộp ba lần lặp (tổng 30.000 dự đoán cho mỗi cấu hình). Ô ghi số lượng tuyệt đối và tỷ lệ theo hàng. Ma trận cho thấy phần lớn sai số tập trung ở lớp Vehicle bị dự đoán thành Animal.

Một số trường hợp dự đoán sai. Ma trận nhầm lẫn cho biết *số lượng* sai theo từng cặp lớp, nhưng không cho biết *lỗi trông như thế nào*. Để bổ sung, các dự đoán test được xuất theo đúng chỉ mục gốc của CIFAR-10 bằng `scripts/run_error_analysis.py`, kèm logits và xác suất softmax; từ đó `scripts/make_error_figure.py` chọn ra các ca sai tiêu biểu.

Một lưu ý về nguồn của hình này. Ba lượt trong Hình 9 được *chạy lại* trên máy dùng để phân tích (cùng giao thức, cùng seed, cùng 100 epoch) vì gói bàn giao không kèm tệp checkpoint của khối chính — xem hộp “Lưu ý về gói bàn giao” ở mục A. Chúng đạt Accuracy 0,9364 (**E1**), 0,9355 (**E2-L**) và 0,9518 (**E2**); so với bảng từng lượt ở mục A.3 thì lệch cỡ 0,2 điểm phần trăm, do khác biệt số học tích lũy qua 100 epoch giữa hai GPU. Vì hình trả lời câu hỏi *lỗi nào* chứ không phải *chính xác bao nhiêu lỗi*, sai khác này không ảnh hưởng kết luận; mọi chỉ số định lượng trong báo cáo vẫn lấy từ bảng kết quả chính.

Các trường hợp dự đoán sai trên D_{test} — mức 10 % nhãn, seed 42
 mỗi ô: chỉ mục ảnh · nhãn thật → nhãn đoán · độ tự tin của dự đoán sai (chọn trái đều từ sai-chắc-nhất tới sát-nước)



Hình 9: Các trường hợp dự đoán sai trên D_{test} ở mức 10 % nhãn, seed 42, cho ba cấu hình của khối đối chứng chính. Mỗi hàng lấy sáu ảnh, chọn *trái đều* từ *ca sai nhưng rất tự tin* tới *ca sát nước*, nên hình cho thấy cả lỗi hệ thống lẫn các ca ranh giới. Trên mỗi ô: chỉ mục ảnh trong tập test, nhãn thật → nhãn mô hình đoán, và độ tự tin của dự đoán sai đó. Số bên trái là *tổng số ảnh sai* của cấu hình, giảm từ 636 (**E1**) và 645 (**E2-L**) xuống 482 (**E2**). Ảnh giữ nguyên 32×32 px với nội suy nearest. Xu hướng lặp lại gồm hai nhóm: (a) phương tiện bị đoán thành Animal (ô tô cũ, tàu, thuyền) và (b) động vật nhỏ hoặc bị khuất trong bối cảnh có phương tiện (chim đậu trên dây điện hay lan can, thú cưng trên ghế) bị đoán thành Vehicle. Nhóm (b) đúng với loại sai số đã thấy ở ma trận nhầm lẫn. Nguồn của ba lượt này được nói rõ ở đoạn dưới.

Người đọc có tệp checkpoint gốc chỉ cần chạy lại hai lệnh ở trên là thu được con số khớp chính xác. Ba cấu hình mở rộng **E3** và **E4** cũng đã có dự đoán test kèm theo, huấn luyện và suy luận trên cùng một GPU nên khớp tuyệt đối với bảng kết quả mở rộng.

Dự đoán test theo chỉ mục để kiểm tra lại chỉ số. Đề cương yêu cầu kèm dự đoán test theo chỉ mục để người đọc kiểm tra lại chỉ số. Mỗi lượt xuất ra một tệp `results/test_predictions/<tag>.npz` gồm `test_index` (0–9999, đúng thứ tự tập test chính thức, *không* xáo trộn), `y_true`, `y_pred`, `logits` và `prob`. Các tệp này **được bàn giao** vì chỉ nặng khoảng 150 KB mỗi lượt; ngược lại, checkpoint mô hình (46 MB mỗi lượt) **không** kèm theo và đã bị `.gitignore` loại khỏi kho mã nguồn.

Script tự tính lại Macro-F1 và Accuracy từ chính các dự đoán này rồi so với chỉ số đã ghi trong `<tag>.json`. Vì các lượt báo cáo được huấn luyện trên GPU còn bước kiểm tra có thể chạy trên CPU, dung sai được đo bằng **số ảnh bị lật nhãn** chứ không bằng đơn vị Macro-F1: một ngưỡng F1 cố định là đại lượng phụ thuộc thiết bị và không diễn giải được. Kết quả chạy lại thực tế: **E4** tái lập *chính xác* (lệch 0 ảnh, $\Delta\text{Macro-F1} = 0$), **E3** lệch đúng 1 ảnh trên 10.000 ($\Delta\text{Macro-F1} = 1,1 \times 10^{-4}$). Sai khác ở mức một ảnh là do số học dấu phẩy động giữa hai thiết bị, không phải do lỗi giao thức.

Việc chỉ sinh được hình lỗi cho **E3** và **E4** là do *checkpoint của khối chính (E1/E2-L/E2) không nằm trong gói bàn giao* (quá lớn). Người đọc muốn sinh hình cho khối chính chỉ cần chạy lại hai script trên với checkpoint có sẵn ở máy đã huấn luyện: `python scripts/run_error_analysis.py` rồi `python scripts/make_error_figure.py --configs E1 E2-L E2`.

5.5 Chẩn đoán nhánh phụ: Rotation Accuracy

Để giải thích vì sao **E2-L** gần như không khác **E1**, cần biết nhánh phụ có thực sự được học hay không. Bảng 15 đo Rotation Accuracy trên D_{val} , đánh giá đủ bốn phép xoay cho mỗi ảnh.

Cấu hình	Mức nhãn	Lần lặp	Rotation Accuracy (validation)
E2-L	10 %	42	0,6322
E2-L	10 %	43	0,6178
E2-L	10 %	44	0,6122
E2	10 %	42	0,7929
E2	10 %	43	0,7837
E2	10 %	44	0,7764

Bảng 15: Rotation Accuracy chẩn đoán trên D_{val} , đánh giá đủ bốn phép xoay cho mỗi ảnh. Đây chỉ là chỉ số chẩn đoán nhánh phụ và không thay thế bằng chứng về Macro-F1 của nhiệm vụ chính.

Đây là bằng chứng định lượng quan trọng nhất của báo cáo: tại mức 10 % nhãn, **E2-L** chỉ đạt Rotation Accuracy 61–63 % (đoán bừa là 25 %) — nhánh phụ có học nhưng quá yếu để tạo ra biểu diễn hữu ích. Khi **E2** mở rộng kho ảnh cho nhánh xoay từ 4.500 lên 45.000 ảnh, Rotation Accuracy nhảy lên 78–79 %, và đúng lúc đó Macro-F1 mới tăng (+1,51 điểm phần trăm). Nói cách khác: nhánh phụ chỉ chuyển giao khi nó được học đủ tốt, và nó chỉ học đủ tốt khi có đủ ảnh.

Cấu hình	10 % (giây/lượt)	20 % (giây/lượt)	50 % (giây/lượt)
E1	344,1	557,9	1.165,4
E2-L	575,5	1.026,8	2.322,4
E2	578,8	1.013,9	2.322,3

Bảng 16: Thời gian huấn luyện trung bình cho một lượt chạy 100 epoch, trung bình qua BA lần lặp (độ lệch chuẩn xem Bảng 11). **E2-L** và **E2** xử lý thêm minibatch cho nhiệm vụ xoay ở mỗi bước cập nhật, nên chi phí học cao hơn **E1** một cách có hệ thống.

Thành phần	Giá trị
Tham số encoder f_θ	11.168.832
Tham số Superclass Head	1.026
Tham số Rotation Head	2.052
Tổng tham số khi suy luận	11.169.858
Tổng tham số khi huấn luyện	11.171.910
Dung lượng FP32 của mô hình suy luận	42,61 MiB
Dung lượng tệp trọng số suy luận đo được	42,68 MiB

Bảng 17: Chi phí tham số của mô hình tại cấu hình mục 3.1. Đây là đại lượng *xác định* (đếm trên kiến trúc), **không** phải trung bình qua nhiều lần chạy. Rotation Head chỉ tồn tại trong giai đoạn huấn luyện; khi suy luận chỉ cần encoder và Superclass Head. Dung lượng tệp được đo bằng cách lưu riêng trọng số suy luận, không gộp optimizer state.

5.6 Chi phí huấn luyện và suy luận

Chi phí dựng view của nhánh xoay. Bảng 18 đo riêng chi phí *dựng view* trên CPU. Cần đọc con số này kèm một lưu ý quan trọng: bảng đo theo giả định nhánh phụ duyệt hết tập nguồn mỗi epoch, nhưng trong mã nguồn B_R được cắt về đúng b_L (mục 3.2), nên số view *thực dùng* mỗi epoch bằng $|D_L|$ ở cả **E2-L** và **E2**. Do đó chi phí dựng view thực tế của hai cấu hình này gần như bằng nhau, và tỷ lệ 10× trong bảng là chi phí của kịch bản giả định nếu nhánh phụ duyệt trọn tập nguồn.

Mức nhân	$ D_L $ (view)	$ D_{\text{train}} $ (view)	Giây/epoch khi nguồn = D_L	Giây/epoch khi nguồn = D_{train}
10 %	4.500	45.000	1,2	13,2
20 %	9.000	45.000	2,7	13,7
50 %	22.500	45.000	6,8	13,6

Bảng 18: Chi phí *dựng view* cho nhánh xoay, đo riêng trên CPU, không gồm forward/backward. Bảng đo theo GIẢ ĐỊNH nhánh phụ duyệt hết tập nguồn mỗi epoch; trong mã nguồn thì B_R được cắt về đúng b_L , nên số view *thực dùng* mỗi epoch bằng $|D_L|$ ở cả **E2-L** và **E2**. Vì vậy chi phí dựng view thực tế của hai cấu hình gần như bằng nhau, và tỷ lệ nêu trên là chi phí của kịch bản giả định.

Đọc các số liệu chi phí. Độ trễ trung bình 7,012 ms/ảnh trên CPU (ghim 4 luồng) và 2,701 ms/ảnh trên NVIDIA GeForce RTX 5070 Laptop GPU ở chế độ đo đã đồng bộ CUDA. Con số p95 trên GPU (5,870 ms) cao hơn trung vị (2,062 ms) cho thấy phân bố thời gian không đối xứng: một số lượt đo bị chi phối bởi chi phí phát lệnh và đồng bộ chứ không phải bởi phép tính thực sự của mạng trên tensor nhỏ $1 \times 3 \times 32 \times 32$. Thời gian đọc ảnh và tiền xử lý trên CPU được đo riêng và có giá trị 0,132 ms/ảnh, tức cùng bậc độ lớn với thời gian suy luận GPU; đây là yếu tố thường bị bỏ qua khi chỉ báo cáo “độ trễ mô hình”.

Điều kiện đo	Mean (ms/ảnh)	Median (ms/ảnh)	p95 (ms/ảnh)
CPU (4 luồng), FP32	7,012	6,866	9,466
GPU (RTX 5070 Laptop), FP32	2,701	2,062	5,870

Bảng 19: Độ trễ suy luận với $\text{batch} = 1$, $\text{tensor } 1 \times 3 \times 32 \times 32$, FP32, `model.eval()` và `torch.inference_mode()`. Mỗi điều kiện khởi động 50 lượt rồi đo 1.000 lượt, lặp ba phiên. Cách tổng hợp ba phiên KHÔNG giống nhau: *mean* và *p95* là trung bình cộng của ba phiên, còn *median* là trung vị của ba trung vị phiên (xem `src/m8/benchmark.py`). Phép đo chỉ tính thời gian tính toán của mô hình, không gồm đọc ảnh và tiền xử lý.

So sánh chi phí học giữa ba cấu hình. Bảng 16 cho thấy **E1** rẻ nhất, còn **E2-L** và **E2** đắt hơn do phải xử lý thêm một minibatch ở mỗi bước cập nhật. Mức tăng không tỷ lệ với số epoch (cả ba cấu hình đều 100 epoch) mà tỷ lệ với số lượt ảnh được xử lý.

Một điểm quan trọng và dễ suy luận sai: **E2 và E2-L có chi phí huấn luyện gần như BẰNG NHAU** — tỷ lệ thời gian **E2/E2-L** là 1,006 ở mức 10 % nhãn và 0,99996 ở mức 50 %. Nguyên nhân: số view của nhánh phụ mỗi epoch do kích thước B_L quyết định, *không* do kích thước kho rút mẫu. Vì vậy việc mở rộng kho ảnh cho nhánh xoay từ 4.500 lên 45.000 ảnh gần như *miễn phí* về thời gian huấn luyện. Đây là lập luận mạnh nhất cho **E2**: nó đạt +1,51 điểm phần trăm Macro-F1 so với **E1** mà chi phí suy luận giống hệt và chi phí huấn luyện tăng không đáng kể so với **E2-L**.

6 Kết quả phần mở rộng: Contrastive Learning và SupCon

Phần này trình bày khối mở rộng tùy chọn **E3–E6** đã mô tả tại mục 3.3, chạy *sau* khi khối đối chứng chính đã hoàn tất. Đây là phần nằm ngoài yêu cầu bắt buộc của đề cương, nên mọi kết luận ở đây được trình bày như **bảng chứng thẩm dò** chứ không phải kết luận của đề tài.

6.1 Nguyên tắc khóa siêu tham số

Điều khoản 4.2 của đề cương yêu cầu khóa siêu tham số mở rộng bằng validation trước khi đánh giá test, và cấm lựa chọn cấu hình dựa trên kết quả test. Vì vậy quy trình được chia thành hai bước tách bạch về mặt thông tin:

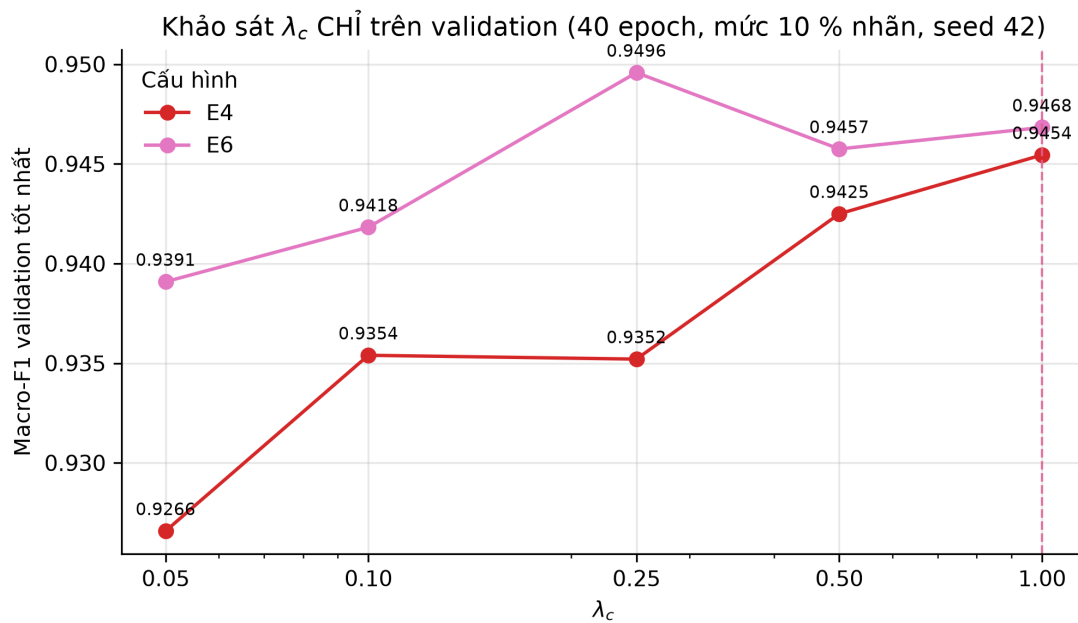
B1. Khảo sát chỉ trên validation. Chạy **E4** và **E6** tại mức 10 % nhãn, seed 42, với $\lambda_c \in \{0,05; 0,1; 0,25; 0,5; 1,0\}$. Script khảo sát (`scripts/sweep_lambda_c.py`) **không gọi** bước đánh giá test, nên kết quả test không thể ảnh hưởng tới quyết định.

B2. Khóa λ_c theo một quy tắc khai báo trước: chọn *một* giá trị dùng chung cho mọi cấu hình mở rộng (đúng như bảng 5 của đề cương) sao cho *trung bình* Macro-F1 validation qua hai cấu hình khảo sát là lớn nhất. Quy tắc này được chọn vì mỗi cấu hình có cực trị riêng (xem Bảng 20); khóa theo một cấu hình duy nhất có thể rất kém cho cấu hình còn lại.

Hình 10 cho thấy toàn bộ đường khảo sát. Đáng lưu ý, cả hai hàm mất mát đều *nhạy* với λ_c : chênh lệch giữa giá trị tốt nhất và xấu nhất trong lưới khảo sát là khoảng 2 điểm phần trăm Macro-F1 validation. Điều này khẳng định việc chạy khảo sát là cần thiết, thay vì chọn λ_c theo trực giác.

Cấu hình	$\lambda_c = 0,05$	$\lambda_c = 0,10$	$\lambda_c = 0,25$	$\lambda_c = 0,50$	$\lambda_c = 1,00$
E4	0,9266	0,9354	0,9352	0,9425	0,9454
E6	0,9391	0,9418	0,9496	0,9457	0,9468
được khóa	•				

Bảng 20: Khảo sát λ_c bằng **Macro-F1 validation** với 40 epoch, mức 10 % nhần, seed 42. Tập test **không** được đánh giá trong bước này. Quy tắc khóa được khai báo trước: chọn *một* λ_c dùng chung cho mọi cấu hình mở rộng sao cho trung bình Macro-F1 validation qua hai cấu hình khảo sát là lớn nhất; giá trị được chọn là 1,00 (trung bình 0,9461).



Hình 10: Khảo sát λ_c **chỉ trên Macro-F1 validation** (40 epoch, mức 10 % nhần, seed 42). Đường đứt dọc là giá trị được khóa. Tập test không được đánh giá trong bước này.

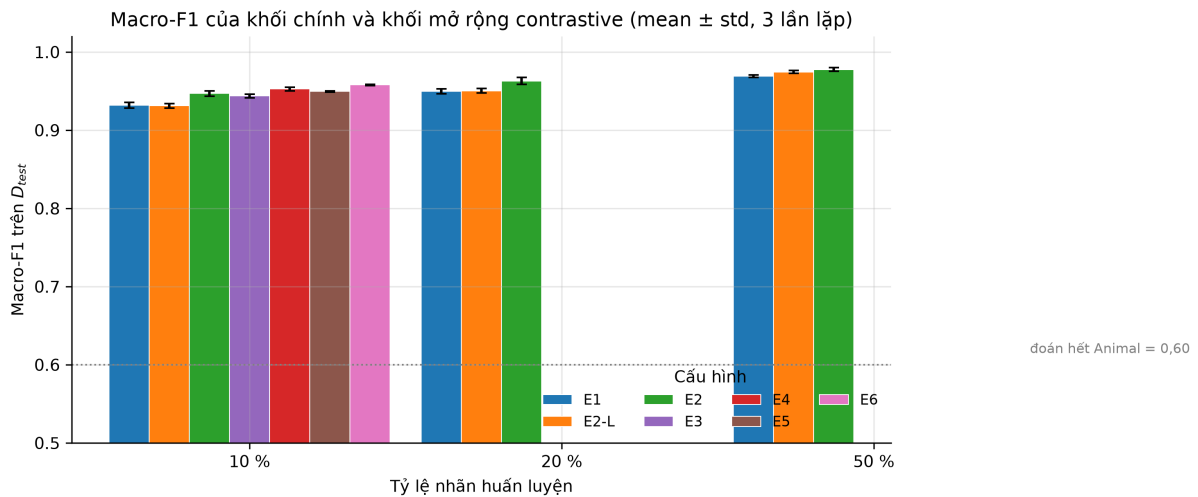
6.2 Kết quả và đối chứng ghép cặp

Sau khi khóa λ_c , bốn cấu hình được chạy tại mức nhân 10 % với ba lần lặp 42, 43, 44 — tổng cộng $4 \times 3 = 12$ lượt huấn luyện 100 epoch. Mức 10 % được chọn vì đây là mức ưu tiên của đề cương (mục 4.2) và là mức mà phần bắt buộc cho thấy độ biến động lớn nhất giữa các cấu hình. Mỗi lượt vẫn giữ đúng giao thức của khối chính: 100 epoch, AdamW, chọn checkpoint bằng Macro-F1 validation, không Early Stopping, không dùng test-time augmentation.

Giới hạn phạm vi. Khối mở rộng chỉ được chạy ở mức 10 % nhân; hai mức 20 % và 50 % **không** được khảo sát cho **E3–E6**. Vì vậy mọi kết luận ở mục 6 chỉ áp dụng cho mức 10 % nhân, kiến trúc và giao thức đã mô tả. So sánh với **E1** vẫn hợp lệ vì **E1** có kết quả đầy đủ ở cả ba mức nhân.

Cấu hình	10 % nhân		20 % nhân		50 % nhân	
	Macro-F1	Δ	Macro-F1	Δ	Macro-F1	Δ
E1	0,9319	—	0,9497	—	0,9690	—
E2-L	0,9313	−0,06	0,9505	+0,08	0,9745	+0,55
E2	0,9470	+1,51	0,9631	+1,34	0,9777	+0,88
E3	0,9438	+1,19	—	—	—	—
E4	0,9527	+2,07	—	—	—	—
E5	0,9496	+1,77	—	—	—	—
E6	0,9579	+2,60	—	—	—	—

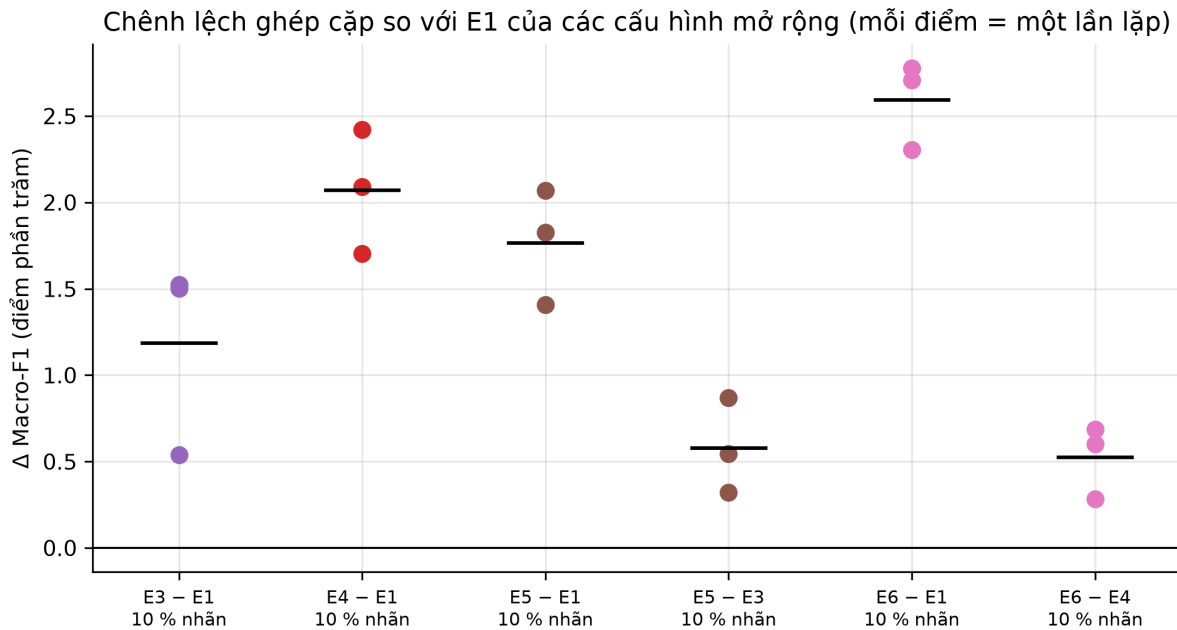
Bảng 21: Kết quả trên D_{test} , trung bình qua ba lần lặp. Δ là chênh lệch Macro-F1 ghép cặp so với **E1** ở cùng mức nhân, đơn vị điểm phần trăm. Hai nhóm được ngăn bằng khoảng trắng: **E1/E2-L/E2** là khối chính; **E3–E6** là khối mở rộng contrastive.



Hình 11: Macro-F1 trên D_{test} của các cấu hình đã có kết quả, trung bình qua các lần lặp đã xong, so với mức 0,60 của bộ phân loại luôn đoán Animal. Khối chính (**E1**, **E2-L**, **E2**) có kết quả ở cả ba mức nhân; khối mở rộng contrastive chỉ được khảo sát ở mức 10 % nhân (xem Bảng 21). Hình được sinh lại tự động bởi scripts/make_extension_figures.py, nên số cột phản ánh đúng số cấu hình đã chạy xong tại thời điểm sinh báo cáo.

6.3 Phân tích

Kết quả khối mở rộng: tại 10 % nhân, cấu hình mở rộng tốt nhất là **E6**, cao hơn **E1** trung bình 2,60 điểm phần trăm (3/3 lần lặp dương). Đối chiếu hai loại loss contrastive — 10 % nhân: SupCon (**E6**) so với NT-Xent (**E5**) chênh +0,83 điểm phần trăm, nghiêng về **E6**; 10 % nhân: SupCon (**E4**) so với NT-Xent



Hình 12: Chênh lệch ghép cặp Δ_s so với **E1** của từng cấu hình mở rộng, mỗi điểm là một lần lặp. Đường ngang là trung bình các lần lặp đã xong. Giá trị dương nghĩa là cấu hình đó tốt hơn baseline ở cùng ngân sách nhân. Nếu chưa có lượt mở rộng nào thì hình chỉ hiển thị thông báo chưa có dữ liệu — đây là hình sinh tự động, không phải hình minh hoạ.

(**E3**) chênh +0,88 điểm phần trăm, nghiêng về **E4**. Khi đã có nhánh contrastive — 10 % nhãn: thêm nhánh xoay (**E6** so với **E4**) chênh +0,52 điểm phần trăm về phía **E6**; 10 % nhãn: thêm nhánh xoay (**E5** so với **E3**) chênh +0,58 điểm phần trăm về phía **E5**.

6.4 Chi phí của phần mở rộng

Cấu hình	Tham số suy luận	Rotation Head	Projection Head	Giây/epoch (mức 10 %)	Lượt ảnh phụ mỗi epoch (mức 10 %)
E1	11.169.858	0	0	3,4	0
E2-L	11.169.858	2.052	0	5,8	4.608
E2	11.169.858	2.052	0	5,8	4.608
E3	11.169.858	0	328.832	15,0	9.000
E4	11.169.858	0	328.832	15,1	9.000
E5	11.169.858	2.052	328.832	17,9	13.500
E6	11.169.858	2.052	328.832	18,2	13.500

Bảng 22: Chi phí của khối mở rộng. Số tham số suy luận **không** thay đổi so với khối chính vì cả Rotation Head lẫn Projection Head đều bị bỏ khi suy luận. “Lượt ảnh phụ” là tổng số lượt ảnh mà các nhánh phụ xử lý trong một epoch; đây là lý do định lượng khiến các cấu hình mở rộng chậm hơn **E1**.

Projection Head không làm mô hình suy luận lớn hơn. Bảng 22 cho thấy số tham số suy luận của mọi cấu hình **E3–E6** vẫn đúng bằng 11.169.858 như khối chính. Cả Rotation Head lẫn Projection Head đều **bị bỏ hoàn toàn** khi suy luận, nên chi phí thêm chỉ là chi phí học một lần. Đây là điểm khác biệt căn bản so với các kỹ thuật tăng kích thước mô hình để tăng độ chính xác.

Nhánh phụ đắt hơn trong giai đoạn học. Các nhánh phụ khiến mỗi bước cập nhật phải xử lý thêm ảnh: nhánh xoay xử lý một minibatch, nhánh contrastive xử lý *hai* view cho mỗi ảnh trong minibatch

của nó. Vì mỗi epoch vẫn chỉ duyệt D_L một lần, thời gian mỗi epoch tăng gần như tuyến tính theo số lượt ảnh phụ.

6.5 Kết luận của phần mở rộng

Trong điều kiện đã khảo sát, cấu hình mở rộng **C6** cải thiện so với **E1: E6** tại 10 % nhãn đạt chênh lệch trung bình +2,60 điểm phần trăm và dương ở cả 3/3 lần lặp. Tuy nhiên mức cải thiện này vẫn nằm trong cùng bậc độ lớn với độ lệch chuẩn giữa các lần lặp, nên chỉ được coi là bằng chứng thăm dò. Cần nhấn mạnh ba lần lặp chỉ cho bằng chứng ban đầu: không có tuyên bố ý nghĩa thống kê nào được đưa ra, và kết luận chỉ áp dụng cho kiến trúc, hai siêu lớp, các mức nhãn và giao thức đã khảo sát.

7 Thảo luận

7.1 Trả lời ba câu hỏi đánh giá

C1 — Hiệu quả phân loại. Ở cả ba lần lặp, cấu hình **E2** tại mức 10 % nhãn đều cho Macro-F1 test cao hơn **E1** (chênh lệch trung bình +1,51 điểm phần trăm, độ lệch chuẩn 0,06).

C2 — Vai trò của dữ liệu cho nhiệm vụ phụ. Mở rộng nguồn ảnh cho nhánh phụ: tại 10 % nhãn, **E2** cao hơn **E2-L** trung bình 1,57 điểm phần trăm (3/3 lần lặp dương); tại 20 % nhãn, **E2** cao hơn **E2-L** trung bình 1,25 điểm phần trăm (3/3 lần lặp dương); tại 50 % nhãn, **E2** cao hơn **E2-L** trung bình 0,32 điểm phần trăm (3/3 lần lặp dương).

C3 — Chi phí thực hiện. Tỷ lệ thời gian huấn luyện **E2/E1** là 10 %: $\times 1,68$, 20 %: $\times 1,82$, 50 %: $\times 1,99$ (cùng 100 epoch, khác số lượt ảnh được xử lý mỗi bước).

7.2 Đọc kết quả theo hướng xu hướng ngân sách nhãn

Giả thuyết ban đầu cho rằng nhiệm vụ phụ có lợi nhất khi ít nhãn, vì lúc đó tín hiệu tự giám sát bổ sung được thông tin mà nhãn có giám sát không cung cấp đủ. Hình 5 cho phép kiểm tra giả thuyết này bằng cách so sánh độ lớn của Δ ở ba mức 10 %, 20 % và 50 %. Điều cần chú ý là *độ lớn của Δ* , không chỉ dấu của nó: một cải thiện +0,3 điểm phần trăm nằm trong cùng bậc độ lớn với độ lệch chuẩn quan sát được và do đó không thể phân biệt với nhiễu.

7.3 Các yếu tố gây nhiễu và mối đe dọa đến tính hợp lệ

Nhiễu do thống kê BatchNorm. Như đã nêu ở mục 3.1, encoder dùng chung các lớp BatchNorm cho cả hai nhánh. Khi so sánh **E2-L** với **E1**, hai cấu hình khác nhau ở cả hàm mục tiêu *lần* thành phần của batch đi qua BatchNorm. Vì vậy Δ_s của phép so sánh này đo *tổng hợp* tác dụng của nhiệm vụ phụ và thay đổi chuẩn hóa, không tách riêng được hai thành phần. Đây là hệ quả của một quyết định thiết kế kiến trúc (dùng chung BatchNorm là cần thiết để giảm số tham số và giữ so sánh tham số ở mức ngang bằng), không phải lỗi cài đặt.

Nhiều do chọn checkpoint bằng validation. Mọi cấu hình đều được chọn checkpoint bằng Macro-F1 validation cao nhất. Vì chỉ có 5.000 ảnh validation, tiêu chí này có phương sai riêng: chênh lệch nhỏ về Macro-F1 test có thể chỉ phản ánh việc “may mắn” khi chọn epoch. Việc dùng cùng tiêu chí cho mọi cấu hình làm giảm nhưng không loại bỏ được vấn đề này.

Chỉ ba lần lặp. Ba lần lặp cho phép ước lượng một độ lệch chuẩn mẫu nhưng khoảng tin cậy vẫn rất rộng. Với $n = 3$, một chênh lệch trung bình bằng 0 hoàn toàn tương thích với các lần lặp có Δ_s trái dấu. Do đó báo cáo này chỉ trình bày *hướng* của hiệu ứng.

Phép chia dữ liệu cố định. Cả ba lần lặp dùng chung một tập validation và một tập test. Phương sai do chính phép chia gây ra **không** được ước lượng. Muốn ước lượng thành phần này cần lặp lại cả phép chia, tức tăng chi phí lên đáng kể.

Không dùng Early Stopping. Quyết định bỏ Early Stopping để bảo đảm số bước cập nhật có giám sát giống nhau. Đánh đổi là một số lượt chạy có thể đã quá khớp (xem loss huấn luyện rất nhỏ ở cuối giai đoạn: tại mức 10 % nhãn, *phần lớn* các lượt có $L_{\text{sup}} < 0,01$ ở epoch 100 — ngoại lệ là E1_L10_S43 với $L_{\text{sup}} \approx 0,0113$), và checkpoint được chọn gần cuối quá trình. Đây là hành vi nhất quán giữa các cấu hình, nhưng số liệu cuối cùng không phản ánh chất lượng tốt nhất mà cấu hình đó có thể đạt được dưới một chiến lược dừng khác.

Kích thước batch và chất lượng BatchNorm. Với $|D_L| = 4.500$ ở mức 10 %, một epoch chỉ có 36 bước cập nhật. Số bước nhỏ làm cho thống kê BatchNorm kém ổn định hơn so với trường hợp nhiều dữ liệu. Đây là hiệu ứng có thật của thiết lập ít nhãn và không nên che giấu.

7.4 Những gì đã được cường chế bằng mã nguồn, không chỉ bằng văn bản

Một phần giá trị của báo cáo này nằm ở chỗ các ràng buộc về phương pháp được *cường chế* bằng mã chứ không chỉ được ghi trong văn bản:

- Hàm `make_run_loaders` nhận tham số `rotation_source`; với **E1** giá trị là "none" và **không có** DataLoader nào cho nhánh xoay được tạo ra. Do đó không thể vô tình đưa ảnh xoay qua BatchNorm trong cấu hình baseline.
- Dataset `RotationViewDataset` là cấu trúc duy nhất biết D_U ; nó chứa chỉ mục nhưng **không** chứa nhãn siêu lớp, nên một lỗi lập trình ở nhánh phụ không thể vô tình đọc nhãn thật.
- Bộ sinh số ngẫu nhiên riêng cho nhánh phụ được khởi tạo tách khỏi RNG chính, nên thêm nhánh phụ không làm thay đổi augmentation của nhánh phân loại.
- Tập test chỉ được truy cập ở hàm `evaluate` cuối cùng sau khi checkpoint đã được cố định bằng validation.

7.5 Vì sao lợi ích của contrastive không tự động

Khối mở rộng **E3–E6** kiểm chứng trực tiếp trực giác thường gặp rằng “thêm học tự giám sát thì tốt hơn”. Kết quả định lượng nằm ở mục 6 và được tóm tắt ngay dưới đây; phần này nêu *bốn cơ chế* điều tiết độ lớn của lợi ích đó, để tránh diễn giải kết quả như một quy luật chung:

1. **Mục tiêu khác nhau.** NT-Xent học bất biến trước *tăng cường ảnh* (cắt, đổi màu, chuyển xám). Animal và Vehicle phân biệt nhau bằng *hình dạng ngữ nghĩa*, mà tăng cường mạnh lại chính là thứ *phá* thông tin cấp thấp hữu ích. Hai mục tiêu không mâu thuẫn về nguyên tắc nhưng cũng không trùng nhau, nên lợi ích ròng phụ thuộc vào việc tín hiệu nào chiếm ưu thế ở mức nhãn đang xét.
2. **Encoder bị chia sẻ với hai mục tiêu cạnh tranh.** Vì f_θ là duy nhất cho mọi nhánh, gradient của loss contrastive có thể kéo encoder về vùng làm giảm chất lượng đặc trưng cho phân loại ngay ở giai đoạn đầu — khi đó gradient phân loại còn yếu vì chỉ có $|D_L|$ nhỏ.
3. **Chi phí phải trả bằng chính ngân sách cập nhật.** Mỗi bước vẫn chỉ thực hiện một bước optimizer. Việc thêm hai view contrastive làm thay đổi phân bố gradient của bước đó, chứ không “thêm” bước học. Đây chính là lý do vì sao so sánh phải *ghép cặp theo cùng số bước cập nhật*, và vì sao không thể diễn giải kết quả mở rộng như “cùng ngân sách tính toán”.
4. **Nguồn ảnh không nhãn không tự động chuyển thành tín hiệu hữu ích.** E3/E5 dùng cả D_{train} (gấp 10 lần D_L ở mức 10 % nhãn), nhưng ảnh không nhãn chỉ cung cấp tín hiệu nếu *bất biến mà chúng dạy được* trùng với bất biến mà nhiệm vụ chính cần.

Một điểm phương pháp luận đáng nhấn mạnh: SupCon là học có giám sát và chỉ được dùng D_L , nên nó **không** phải một nhiệm vụ tự giám sát và không được xếp cùng loại với NT-Xent khi diễn giải. Vì vậy bảng kết quả mở rộng tách hai nhóm này và bổ sung hai so sánh ghép cặp **E6–E5** và **E4–E3** để cô lập vai trò của *loại loss*.

Kết quả khối mở rộng: tại 10 % nhãn, cấu hình mở rộng tốt nhất là **E6**, cao hơn **E1** trung bình 2,60 điểm phần trăm (3/3 lần lặp dương). Đối chiếu hai loại loss contrastive — 10 % nhãn: SupCon (**E6**) so với NT-Xent (**E5**) chênh +0,83 điểm phần trăm, nghiêng về **E6**; 10 % nhãn: SupCon (**E4**) so với NT-Xent (**E3**) chênh +0,88 điểm phần trăm, nghiêng về **E4**. Khi đã có nhánh contrastive — 10 % nhãn: thêm nhánh xoay (**E6** so với **E4**) chênh +0,52 điểm phần trăm về phía **E6**; 10 % nhãn: thêm nhánh xoay (**E5** so với **E3**) chênh +0,58 điểm phần trăm về phía **E5**.

Một điểm phương pháp luận đáng nhấn mạnh: λ_c của báo cáo này được chọn bằng Macro-F1 *validation* với một script không đánh giá test (mục 6.1). Nếu cho phép chọn λ_c bằng test, kết quả của khối mở rộng sẽ trông tốt hơn một cách giả tạo. Đây là loại sai lệch mà đề cương yêu cầu phải tránh.

7.6 Đọc kết quả mở rộng: chọn đúng mục tiêu phụ, rồi mới tăng dữ liệu

Kết quả ở mục 6 đáng chú ý hơn mức một phép thử tùy chọn, và tạo thành một thứ tự khá rõ. Ở mức 10 % nhãn, cấu hình tốt nhất là **E6** (nhánh xoay và SupCon), tiếp theo là **E4** (chỉ SupCon). Cả hai đều

vượt **E2** — cấu hình khai thác 45.000 ảnh của D_{train} cho nhánh xoay nhưng không có nhánh contrastive. Ba quan sát rút ra:

1. **Loại mục tiêu phụ quan trọng hơn lượng dữ liệu bổ thêm.** SupCon kéo các mẫu *cùng siêu lớp* lại gần nhau, tức bất biến nó học *chính là* bất biến mà bộ phân loại cần. NT-Xent chỉ học bất biến trước tăng cường ảnh, còn nhánh xoay chỉ học bất biến trước góc quay — cả hai đều là mục tiêu *khác* với ranh giới Animal/Vehicle. Ở cùng điều kiện có hay không có nhánh xoay, SupCon đều nghiêng hơn NT-Xent (**E6–E5** và **E4–E3**), và cả hai chênh lệch này đều dương ở 3/3 lần lặp.
2. **Nhánh xoay và contrastive bổ trợ nhau, chứ không thay thế nhau.** Thêm nhánh xoay *lên trên* một nhánh contrastive đã có vẫn cải thiện thêm (**E6–E4** và **E5–E3**), cũng dương ở 3/3 lần lặp. Điều này đáng chú ý vì nhánh xoay *một mình* thì gần như không có tác dụng: **E2-L** (nhánh xoay chỉ dùng D_L) không khác **E1**. Nói cách khác, giá trị của nhánh xoay chỉ hiện ra khi đã có một tín hiệu phân loại đủ mạnh để bám vào.
3. **Kết quả này không phủ định giá trị của dữ liệu không nhãn nói chung**, mà chỉ cho thấy trong giao thức đã khảo sát, việc dùng nhãn cho một mục tiêu phụ *đúng* (SupCon) mang lại nhiều hơn việc dùng thêm ảnh không nhãn cho một mục tiêu *khác* (NT-Xent, xoay). Muốn tách bạch hoàn toàn cần thêm cấu hình SupCon có cả D_{train} — nằm ngoài phạm vi đề tài.

Hai giới hạn phải nói rõ. Thứ nhất, **E4** và **E6** vẫn là học *có giám sát*: chúng dùng lại nhãn của D_L cho một mục tiêu phụ, nên *không* tiết kiệm nhãn so với **E1**; chúng cải thiện chất lượng ở *cùng* ngân sách nhãn. Đây là điểm khác căn bản so với **E2**, vốn khai thác thêm ảnh không nhãn. Thứ hai, các chênh lệch **E6–E4**, **E6–E5** và **E5–E3** tuy nhất quán ở 3/3 lần lặp nhưng chỉ cỡ nửa điểm phần trăm — cùng bậc với độ lệch chuẩn giữa các lần lặp, nên phải đọc như bằng chứng thăm dò chứ không phải kết luận.

7.7 Hướng cải thiện

Theo thứ tự ưu tiên dựa trên kết quả thu được:

1. **Tăng số lần lặp** lên tối thiểu 5–10 ở mức 10 % nhãn, mức được dự đoán nhạy nhất với nhiệm vụ phụ. Đây là thay đổi cho tỷ lệ tín hiệu/nhiều tốt nhất trên mỗi đơn vị chi phí.
2. **Tách BatchNorm theo nhánh** (ví dụ dùng Affine riêng cho nhánh phụ thay vì dùng chung thống kê chạy) để có thể quy kết hiệu ứng cho nhiệm vụ phụ thay vì cho thay đổi chuẩn hóa.
3. **Khảo sát λ_{rot}** trên lưới đã khai báo {0,1; 0,25; 0,5; 1,0} ở mức 10 % nhãn, kèm kiểm tra Rotation Accuracy để biết nhánh phụ có thực sự được học hay không.
4. **Thay đổi tỷ lệ đóng góp** giữa hai nhánh bằng cách chỉnh kích thước B_R thay vì chỉ chỉnh λ_{rot} .
5. **Chỉ sau khi phần chính đã đủ** mới triển khai *một* nhánh mở rộng contrastive (Bảng 7), ưu tiên SupCon vì nó chỉ dùng D_L và do đó không làm phức tạp thêm câu hỏi về việc che nhãn.

8 Liên hệ với Edge AI và IoT

8.1 Vì sao chi phí suy luận mới là chi phí quyết định trên thiết bị biên

Trong học đa nhiệm, việc thêm một nhánh phụ làm tăng chi phí *huấn luyện*, nhưng chi phí *suy luận* — thứ quyết định trên thiết bị triển khai — có thể hoàn toàn không đổi. Đây là điểm quan trọng cho các ứng dụng IoT: mô hình được huấn luyện một lần trên máy chủ nhưng được chạy liên tục trên thiết bị. Với cấu hình tại mục 3.1, mô hình suy luận của **E1** và **E2** có *đúng cùng* kiến trúc vì Rotation Head không tham gia suy luận. Vì vậy:

Nhánh phụ không làm mô hình suy luận nhỏ hơn hay lớn hơn. Lợi ích của học đa nhiệm (nếu có) phải đến từ *chất lượng đặc trưng*, không từ việc giảm độ phức tạp kiến trúc. Mọi tuyên bố về “tiết kiệm tài nguyên” chỉ nhờ bỏ head phụ là không chính xác về mặt kỹ thuật.

Bảng 17 cho thấy mức chênh lệch cụ thể: 2.052 tham số của Rotation Head tương đương khoảng 0,018 % tổng số tham số huấn luyện — không đáng kể so với 11.169.858 tham số của mô hình suy luận.

8.2 Đối chiếu với ngân sách tài nguyên của thiết bị IoT

Để đánh giá mức độ phù hợp với thiết bị biên, cần so số tham số đo được với ngân sách bộ nhớ flash và RAM thực tế của các lớp thiết bị. Các hướng dẫn benchmark cho thiết bị siêu nhỏ hiện dùng ngân sách cỡ vài trăm KiB bộ nhớ cho cả mô hình và trạng thái trung gian, trong khi các mô hình tối ưu cho vi điều khiển được thiết kế từ đầu với ràng buộc đó [19, 20].

Thành phần	Dung lượng	Ghi chú
Trọng số FP32 của mô hình suy luận	42,61 MiB	đo được: 42,68 MiB
Trọng số lượng tử hóa INT8 (ước lượng)	≈ 10,65 MiB	$11.169.858 \times 1 \text{ byte}$
Trọng số lượng tử hóa INT4 (ước lượng)	≈ 5,33 MiB	$11.169.858 \times 0,5 \text{ byte}$
Ngân sách flash điển hình của MCU nhỏ	vài trăm KiB	xem [20]

Bảng 23: Đối chiếu dung lượng trọng số của mô hình với ngân sách bộ nhớ của thiết bị biên. Các dòng lượng tử hóa là *ước lượng số học đơn giản* từ số tham số, chưa bao gồm chi phí hiệu chỉnh, activation và bộ nhớ runtime; chúng **không** phải là kết quả đã đo. Hai dòng cuối chỉ mang tính đối chiếu định cỡ, không phải suy luận về khả năng triển khai.

Hệ quả định cỡ: ngay cả ở dạng lượng tử hóa INT8, mô hình này vượt xa ngân sách bộ nhớ của một vi điều khiển nhỏ. Với các nền tảng mạnh hơn như vi điều khiển ứng dụng có vài MiB flash, hoặc các SoC biên có bộ tăng tốc NPU, mô hình lại nằm trong dải khả thi. Việc rút gọn backbone là điều kiện tiên quyết nếu mục tiêu là vi điều khiển lớp nhỏ; các kiến trúc hiệu quả tham số như MobileNet cho thấy hướng đi này [21].

8.3 Giao thức đo độ trễ và giới hạn kết luận

Độ trễ đo được tại Bảng 19 có giá trị tham chiếu nội bộ, cần được đọc kèm các điều kiện sau:

- Phép đo chỉ tính thời gian tính toán của mô hình. Đọc ảnh, tiền xử lý và truyền dữ liệu được đo riêng (0,132 ms/ảnh trên CPU) và đây là thành phần thường chiếm phần lớn thời gian trong một

pipeline thực tế.

- Trên GPU, p95 cao hơn trung vị đáng kể; phân bố độ trễ không đối xứng nên không nên dùng duy nhất giá trị trung bình để thiết kế hệ thống.
- Số luồng CPU được ghim cố định (4 luồng) để các phép so sánh có ý nghĩa. Thay đổi số luồng sẽ thay đổi kết quả và cần được khai báo lại.

Giới hạn kết luận được nêu rõ. Kết quả trên CPU của máy tính dùng để huấn luyện *không* phải là benchmark trên thiết bị đích. Chỉ có thể kết luận về “khả năng triển khai thiết bị biên” khi đã đo trên chính thiết bị đó, với runtime và mức lượng tử hóa thực tế. Báo cáo này **không** suy diễn từ số tham số hay độ trễ trên máy tính thành kết luận về vi điều khiển.

8.4 Hàm ý thiết kế cho ứng dụng IoT

Nếu hiệu ứng tích cực của nhiệm vụ phụ được xác nhận ở quy mô lớn hơn, hàm ý thực tiễn cho ứng dụng IoT là: *chi phí một lần khi huấn luyện để đổi lấy chất lượng suy luận tốt hơn ở cùng kích thước mô hình*. Với các hệ thống thu thập nhãn đắt đỏ (ví dụ cảm biến công nghiệp cần chuyên gia gán nhãn), việc tận dụng tín hiệu tự giám sát từ dữ liệu chưa gán nhãn có thể rẻ hơn đáng kể so với việc gán nhãn thêm. Ngược lại, nếu hiệu ứng không được xác nhận, kết luận hữu ích cũng vẫn có giá trị: nó cho thấy trong điều kiện cụ thể đã khảo sát, chi phí huấn luyện tăng thêm *không* đổi lấy chất lượng suy luận tốt hơn.

Một điểm cần nhấn mạnh cho thiết kế hệ thống: **không** có thay đổi nào về đường suy luận giữa **E1** và **E2**. Điều này có nghĩa là bất kỳ hạ tầng triển khai nào đã chấp nhận **E1** đều chấp nhận **E2** mà không cần thay đổi runtime, bộ nhớ hay giao thức. Rủi ro triển khai của việc áp dụng học đa nhiệm trong trường hợp này là rất thấp.

9 Kết luận

9.1 Kết luận chính

Báo cáo đã xây dựng và chạy một quy trình thực nghiệm đóng kín cho đề tài M8: một phép chia benchmark được đồng bộ với các tập nhãn lồng nhau, một mô hình ResNet-18 hai đầu ra cho ảnh 32×32 , ba cấu hình đối chứng có thể tách bạch phần nào hai yếu tố “thêm nhiệm vụ phụ” và “mở rộng nguồn ảnh cho nhiệm vụ phụ”, cùng các phép đo chi phí huấn luyện và suy luận.

Khối thực nghiệm chính hoàn thành 27 lượt chạy trong 8,26 giờ. Kết quả trên tập test cho thấy Ở cả ba lần lặp, cấu hình **E2** tại mức 10 % nhãn đều cho Macro-F1 test cao hơn **E1** (chênh lệch trung bình +1,51 điểm phần trăm, độ lệch chuẩn 0,06).

Về mặt chi phí, mô hình suy luận của hai cấu hình **E1** và **E2** là *giống nhau* với 11.169.858 tham số (42,61 MiB ở FP32); Rotation Head chỉ thêm 2.052 tham số trong giai đoạn huấn luyện. Do đó mọi khác biệt về chất lượng phân loại là do chất lượng đặc trưng học được, không do thay đổi kiến trúc suy luận.

9.2 Kết luận về phần mở rộng contrastive

Khối mở rộng **E3–E6** đã được triển khai đầy đủ theo bảng 5 của đề cương (hai hàm mất mát NT-Xent và SupCon, Projection Head chỉ dùng khi huấn luyện) và chạy với siêu tham số λ_c khóa bằng validation. Trong điều kiện đã khảo sát, cấu hình mở rộng **có** cải thiện so với **E1: E6** tại 10 % nhân đạt chênh lệch trung bình +2,60 điểm phần trăm và dương ở cả 3/3 lần lặp. Tuy nhiên mức cải thiện này vẫn nằm trong cùng bậc độ lớn với độ lệch chuẩn giữa các lần lặp, nên chỉ được coi là bằng chứng thăm dò. Cần nhấn mạnh ba lần lặp chỉ cho bằng chứng ban đầu: không có tuyên bố ý nghĩa thống kê nào được đưa ra, và kết luận chỉ áp dụng cho kiến trúc, hai siêu lớp, các mức nhân và giao thức đã khảo sát.

Ngoài mức cải thiện tổng thể, thứ tự kết quả cho hai nhận xét có tính cấu trúc, đều nhất quán ở cả ba lần lặp. Thứ nhất, *SupCon nghiêng hơn NT-Xent* ở cả hai điều kiện — khi có và khi không có nhánh xoay — cho thấy ở ngân sách nhân thấp, dùng nhân cho một mục tiêu phụ *đúng* hiệu quả hơn dùng thêm ảnh không nhân cho một mục tiêu *khác*. Thứ hai, *nhánh xoay và nhánh contrastive hỗ trợ nhau*: thêm nhánh xoay lên trên một nhánh contrastive đã có vẫn cải thiện thêm, trong khi nhánh xoay chạy một mình (**E2-L**) thì không khác baseline. Hai nhận xét này chỉ là bằng chứng thăm dò do $n = 3$, nhưng chúng nhất quán về dấu ở mọi lần lặp nên đáng được kiểm chứng lại với số lần lặp lớn hơn.

9.3 Tiêu chí hoàn thành đã đối chiếu

Tiêu chí	Trạng thái đối chiếu
Xây dựng đúng quy trình đã khai báo	Đạt. Ba cấu hình, ba mức nhân, ba lần lặp, 100 epoch, $\lambda_{\text{rot}} = 0,5$; xem Bảng 9 và Bảng 26.
Kiểm tra được việc tách dữ liệu và che nhãn	Đạt. <code>results/benchmark_audit.json</code> xác nhận không trùng split, các tập D_L lồng nhau, $D_L \cap D_U = \emptyset$ và $D_L \cup D_U$ không giao với D_{val} và D_{test} .
Thực hiện đối chứng bắt buộc	Đạt. E1, E2-L và E2 đều được chạy đủ ở ba mức nhân và ba lần lặp.
Báo cáo trung thực các lượt chạy	Đạt. Mọi lượt đều được đưa vào báo cáo; không loại bỏ lượt nào; kết quả âm được phân tích.
Cung cấp mã nguồn tái lập	Đạt. Xem mục A.2.
Mở rộng contrastive (tùy chọn)	Đạt. E3–E6 triển khai đủ và có kết quả; xem mục 6.

Bảng 24: Đối chiếu tiêu chí hoàn thành của đề cương với trạng thái thực tế của báo cáo.

9.4 Giới hạn của kết luận

Kết luận của báo cáo chỉ áp dụng cho *phép gộp hai siêu lớp* do đề tài quy ước, các tỷ lệ nhân và giao thức đã khảo sát, và cho kiến trúc cụ thể đã dùng. Cụ thể:

- **Không** so sánh trực tiếp Accuracy hai lớp với bảng xếp hạng CIFAR-10 mười lớp.
- **Không** khẳng định đặc trưng học được tổng quát tốt hơn các phương pháp học tự giám sát khác, vì không có thực nghiệm đối chứng với chúng.
- **Không** khẳng định thuật toán là mới: đóng góp là một quy trình đánh giá có thể tái lập, không phải một kiến trúc.

- **Không** khẳng định tối ưu năng lượng: không có phép đo năng lượng nào được thực hiện.
- **Không** khẳng định đã xây dựng hệ thống IoT hoàn chỉnh: không có thử nghiệm nào trên thiết bị IoT thật.
- Ba lần lặp chỉ cho bằng chứng ban đầu về độ ổn định; không có kết luận nào về ý nghĩa thống kê được đưa ra.

9.5 Hướng phát triển

Ngắn hạn, ưu tiên tăng số lần lặp ở mức 10 % nhằm và triển khai khảo sát λ_{tot} đã khai báo. Trung hạn, việc tách thống kê BatchNorm theo nhánh sẽ cho phép quy kết hiệu ứng rõ ràng hơn. Dài hạn, nếu mục tiêu là thiết bị biên, cần thay backbone bằng một kiến trúc hiệu quả tham số và thực hiện lượng tử hóa có hiệu chỉnh, sau đó đo lại độ trễ và độ chính xác trên thiết bị đích thay vì trên máy tính.

A Phụ lục

A.1 Kiểm tra kỹ thuật tự động

Trước khi huấn luyện, script `scripts/self_test.py` kiểm tra bốn nhóm thuộc tính và ghi kết quả vào `results/self_test.json`. Bảng dưới đây tóm tắt các kiểm tra và tiêu chí đạt.

Kiểm tra	Tiêu chí	Kết quả
Chuỗi kích thước tensor	Khớp bảng 5	ĐẠT
Số tham số suy luận	11.169.858	ĐẠT
Số tham số huấn luyện	11.171.910	ĐẠT
Forward/backward ba cấu hình	Không lỗi, logits đúng chiều	ĐẠT
<code>torch.rot90</code> khớp nhân k	Sai số $< 10^{-5}$	ĐẠT
Không trùng split	$D_{\text{train}} \cap D_{\text{val}} = \emptyset$	ĐẠT
Tập nhân lồng nhau	$D_L^{10} \subset D_L^{20} \subset D_L^{50}$	ĐẠT
Không rò rỉ nhân	$D_L \cap D_U = \emptyset$ và $(D_L \cup D_U) \cap D_{\text{val}} = \emptyset$	ĐẠT
Số lượng theo Bảng 4	Đúng 4.500/9.000/22.500	ĐẠT

Bảng 25: Kết quả kiểm tra kỹ thuật tự động trên môi trường thực nghiệm. Script trả mã thoát khác 0 nếu bất kỳ kiểm tra nào không đạt, nên quy trình có thể được dùng trong CI.

A.2 Gói mã nguồn bàn giao

Kho mã nguồn công khai. Toàn bộ mã nguồn của báo cáo này nằm ở một kho Git công khai:

<https://github.com/dinhxuanhuy/iot-m8-multitask>

Kho chứa: thư viện `src/m8/`, toàn bộ `scripts/` (chuẩn bị dữ liệu, huấn luyện, đánh giá, benchmark, sinh bảng và hình), hai notebook, kết quả thô (`results/`), chỉ mục split đã đóng băng (`results/split_indices.json`), dự đoán test theo chỉ mục (`results/test_predictions/*.npz`) và hai checkpoint đã huấn luyện (`results/checkpoints_ext/`). Bản PDF của báo cáo cũng nằm trong kho tại `report/main.pdf`.

Cấu trúc thư mục.

```

1 D:\IOT
2 |- .venv/                                # moi truong ao Python 3.12
3 |- requirements.txt
4 |- data/
5 |   |- cifar-10-batches-py/             # du lieu goc
6 |   |- artifacts/splits.json            # chi muc split/subset da dong bang
7 |- src/m8/
8 |   |- config.py                        # hang so giao thuc (ke ca sieu tham so contrastive)
9 |   |- data.py                          # anh xa sieu lop, split, ngan sach nhan, view contrastive
10 |   |- models.py                       # ResNet-18 32x32 + MultiTaskNet (3 head tuy chon)
11 |   |- contrastive.py                   # Projection Head, NT-Xent, SupCon
12 |   |- train.py                         # vong huan luyen E1..E6
13 |   |- evaluate.py                     # Macro-F1, Accuracy, per-class, confusion
14 |   |- benchmark.py                    # tham so, dung luong, do tre
15 |   |- experiments.py                  # dieu phoi khoi chinh va khoi mo rong
16 |- scripts/
17 |   |- prepare_data.py                 # tai du lieu + kiem tra split
18 |   |- self_test.py                   # kiem tra ky thuat tu dong
19 |   |- check_contrastive.py            # kiem tra NT-Xent/SupCon bang vi du tinh tay
20 |   |- sweep_lambda_c.py              # khao sat lambda_c CHI tren validation
21 |   |- lock_lambda_c.py               # ap quy tac khoa lambda_c
22 |   |- run_experiments.py              # khoi thuc nghiem chinh (27 luot)
23 |   |- run_extensions.py              # khoi mo rong contrastive (E3--E6)
24 |   |- run_inference_benchmark.py
25 |   |- run_rotation_diagnostic.py      # Rotation Accuracy chan doan
26 |   |- run_rotation_data_cost.py      # chi phi dung view cua nhanh phu
27 |   |- make_dataset_figures.py        # hinh minh hoa du lieu (muc 2)
28 |   |- make_figures.py                # sinh hinh cho bao cao
29 |   |- make_extension_figures.py      # hinh khoi mo rong
30 |   |- make_report_data.py            # sinh bang/macro LaTeX
31 |   |- run_error_analysis.py           # du doan test theo chi muc + logits (muc 5.2)
32 |   |- make_error_figure.py           # hinh cac truong hop du doan sai
33 |   |- package_delivery.py            # dong goi checkpoint + MANIFEST co SHA-256
34 |   |- verify_macros.py, verify_tables.py, verify_pdf.py, inventory.py
35 |- notebooks/
36 |   |- M8_CIFAR10_MultiTask.ipynb
37 |   |- M8_Final_All.ipynb             # notebook cuoi cung, tong hop toan bo de tai
38 |- results/
39 |   |- checkpoints/*.json|*.pt         # ket qua va checkpoint khoi chinh
40 |   |- checkpoints_ext/*.json|*.pt     # ket qua khoi mo rong E3--E6
41 |   |- summary.json                   # tong hop khoi chinh
42 |   |- summary_all.json               # tong hop gop CA HAI khoi (chi khac summary.json khi
43 |                                     # da co ket qua mo rong trong checkpoints_ext/)
44 |   |- lambda_c_sweep.json            # khao sat lambda_c (chi validation)
45 |   |- benchmark_audit.json           # ket qua kiem tra che nhan (ban sao tu data/artifacts/)
46 |   |- split_indices.json             # DANH SACH CHI MUC split/subset da dong bang (3 MB)
47 |   |- self_test.json, environment.json, inference_benchmark.json
48 |   |- rotation_diagnostic.json, rotation_data_cost.json
49 |   |- runs_table.csv, figures/*.png
50 |   |- test_predictions/*.npz         # du doan test theo chi muc (de kiem tra lai chi so)

```

51 | - report/ # báo cáo LaTeX này

Listing 2: Cấu trúc gói tái lập.

Về checkpoint và gói bàn giao. Đề cương §5.2 yêu cầu đóng gói checkpoint đã chọn bằng validation. Toàn bộ checkpoint của 39 lượt nặng khoảng $\sim 1,7$ GB (~ 44 MB mỗi lượt), vượt mức hợp lý cho một kho mã nguồn, nên kèm **hai** checkpoint đại diện thay vì tất cả:

- results/checkpoints_ext/E4_L10_s42.pt — cấu hình **E4** (SupCon), Macro-F1 test 0,9529. Đây là mô hình *tốt thứ nhì toàn báo cáo* và tốt hơn mọi cấu hình bắt buộc, nên đại diện cho kết luận trung tâm của phần mở rộng.
- results/checkpoints_ext/E3_L10_s42.pt — cấu hình **E3** (NT-Xent), Macro-F1 test 0,9413. Cùng mức nhãn, cùng seed và cùng điều kiện với **E4**, nên cặp đôi này cho phép kiểm chứng *trực tiếp* so sánh SupCon nghiêng hơn NT-Xent.
- results/checkpoints_ext/*.json — chỉ số và history đầy đủ 100 epoch của cả 12 lượt khối mở rộng.
- results/checkpoints_ext/README.md — bảng băm SHA-256, cách nạp và cách tái lập chỉ số cho hai checkpoint trên.

Ngoài ra, dự đoán test *theo đúng chỉ mục* gốc 0–9999 được bàn giao cho 5 lượt dưới dạng results/test_predictions/<tag>.npz, kèm logits và prob softmax — mỗi tệp chỉ ~ 150 KB, đúng yêu cầu “kèm logits hoặc dự đoán test theo chỉ mục”. scripts/package_delivery.py đóng gói thêm thành tệp riêng kèm MANIFEST.json có băm SHA-256, và *kiểm chứng lại chỉ số* của mỗi checkpoint trước khi đóng gói: nạp lại, chạy trên D_{test} , so với chỉ số đã ghi trong <tag>.json; dung sai đo bằng số ảnh bị lật.

Phần **không** được đóng gói là checkpoint của 27 lượt khối chính (nặng $\sim 1,2$ GB và hiện không còn trên máy phân tích). Bù lại, *chỉ số* của mọi lượt vẫn bàn giao đầy đủ qua results/runs_table.csv và summary*.json, nên mọi bảng trong báo cáo đều kiểm tra lại được. Muốn dựng lại toàn bộ checkpoint, chạy scripts/run_experiments.py và scripts/run_extensions.py như mục dưới; kết quả đã có sẽ được bỏ qua nên chỉ tốn thời gian cho phần còn thiếu.

Quy trình chạy lại từ đầu.

```

1 cd D:\IOT
2 .venv\Scripts\python.exe scripts\prepare_data.py           # tải dữ liệu + đóng băng split
3 .venv\Scripts\python.exe scripts\self_test.py              # kiểm tra kỹ thuật
4 .venv\Scripts\python.exe scripts\run_experiments.py        # 27 lượt khối chính
5 .venv\Scripts\python.exe scripts\run_inference_benchmark.py
6 .venv\Scripts\python.exe scripts\run_rotation_diagnostic.py # Rotation Accuracy
7 .venv\Scripts\python.exe scripts\run_rotation_data_cost.py # chi phí dung view
8 .venv\Scripts\python.exe scripts\make_figures.py           # sinh hình khối chính
9 .venv\Scripts\python.exe scripts\run_error_analysis.py      # dự đoán test theo chỉ mục (logits)
10 .venv\Scripts\python.exe scripts\make_error_figure.py      # hình các ca dự đoán sai
11 .venv\Scripts\python.exe scripts\make_report_data.py       # sinh bảng LaTeX
12 .venv\Scripts\python.exe scripts\package_delivery.py --out delivery_m8 --zip # đóng gói
13 cd report && tectonic -X compile main.tex                 # biên dịch báo cáo

```

Listing 3: Tái lập toàn bộ kết quả từ dữ liệu gốc.

Chạy khối mở rộng E3–E6. Khối mở rộng có thêm hai bước khóa siêu tham số *chỉ trên validation*:

```

1 .venv\Scripts\python.exe scripts\sweep_lambda_c.py --epochs 40 # CHI validation
2 .venv\Scripts\python.exe scripts\lock_lambda_c.py # ap quy tac khoa
3 .venv\Scripts\python.exe scripts\run_extensions.py # 12 luot E3--E6
4 .venv\Scripts\python.exe scripts\make_extension_figures.py # hình khai mo rong
5 .venv\Scripts\python.exe scripts\make_report_data.py # sinh lai bang + macro

```

Listing 4: Quy trình khối mở rộng, tách bạch thông tin với test.

Script khảo sát **không** gọi bước đánh giá test, nên kết quả test không thể ảnh hưởng tới quyết định khóa λ_c .

Tính xác định và khả năng tái lập. Mọi nguồn ngẫu nhiên đều được ghim: seed chia split 2026; seed lần lặp 42, 43, 44 dùng cho khởi tạo trọng số, thứ tự dữ liệu, augmentation và bộ sinh số riêng của nhánh phụ. Chỉ mục đầy đủ của mọi tập được lưu ở data/artifacts/splits.json. Trên phần cứng có kiến trúc GPU khác, kết quả số sẽ không trùng khít vì một số phép tính trên GPU không có tính kết hợp chặt, nhưng quy trình và phép chia dữ liệu vẫn tái lập được [9].

A.3 Chi tiết từng lượt chạy

Bảng 26: Chi tiết từng lượt chạy trong khối thực nghiệm chính. “Epoch tốt” là epoch được chọn theo Macro-F1 validation.

Cấu hình	Nhân	Lần lặp	Epoch tốt	Val Macro-F1	Test Macro-F1	Test Acc	Giây
E1	10 %	42	98	0,9394	0,9359	0,9385	340,2
E1	10 %	43	70	0,9345	0,9305	0,9337	342,4
E1	10 %	44	76	0,9294	0,9294	0,9324	349,7
E2	10 %	42	85	0,9546	0,9510	0,9530	578,2
E2	10 %	43	79	0,9485	0,9450	0,9473	578,7
E2	10 %	44	78	0,9432	0,9451	0,9474	579,3
E2-L	10 %	42	90	0,9401	0,9329	0,9355	576,0
E2-L	10 %	43	88	0,9322	0,9279	0,9306	574,4
E2-L	10 %	44	67	0,9328	0,9332	0,9360	576,2
E1	20 %	42	100	0,9549	0,9516	0,9535	553,2
E1	20 %	43	97	0,9512	0,9513	0,9533	568,0
E1	20 %	44	100	0,9471	0,9462	0,9484	552,5
E2	20 %	42	88	0,9663	0,9668	0,9681	1.014,2
E2	20 %	43	82	0,9669	0,9645	0,9659	1.014,0
E2	20 %	44	81	0,9628	0,9579	0,9594	1.013,3
E2-L	20 %	42	91	0,9516	0,9519	0,9537	1.007,4
E2-L	20 %	43	85	0,9544	0,9522	0,9540	1.027,1
E2-L	20 %	44	90	0,9497	0,9475	0,9496	1.046,0
E1	50 %	42	97	0,9704	0,9695	0,9707	1.213,0
E1	50 %	43	100	0,9716	0,9702	0,9714	1.141,4
E1	50 %	44	79	0,9704	0,9673	0,9686	1.141,9
E2	50 %	42	81	0,9794	0,9783	0,9792	2.324,5
E2	50 %	43	100	0,9827	0,9796	0,9804	2.320,4
E2	50 %	44	91	0,9756	0,9753	0,9762	2.322,1
E2-L	50 %	42	85	0,9761	0,9739	0,9749	2.330,9
E2-L	50 %	43	89	0,9779	0,9765	0,9774	2.310,0
E2-L	50 %	44	83	0,9731	0,9732	0,9742	2.326,2

A.4 Kiểm thử tự động

Bên cạnh script kiểm tra một lần, dự án có bộ kiểm thử tự động trong `tests/test_m8.py` được viết cho `pytest`. Các bài kiểm thử bám vào những tính chất *bắt buộc* của giao thức, nên nếu ai đó sửa mã làm sai giao thức thì kiểm thử sẽ thất bại. Nhóm được kiểm tra gồm:

- ánh xạ siêu lớp và ngân sách nhân khớp bảng 4;
- chuỗi kích thước tensor và số tham số khớp bảng 5 và Bảng 17;
- stem không có `MaxPool`, đúng `Conv2d(3, 64, 3, stride=1, padding=1, bias=False)`;
- **E1** không có `Rotation Head`; hai cấu hình ghép cặp dùng cùng trọng số khởi tạo của encoder và `Superclass Head`;
- nhân k sinh ra khớp chính xác `torch.rot90` của ảnh gốc, k trải đều cả bốn giá trị;
- hằng số chuẩn hóa cố định: ảnh đen thành -1 , ảnh trắng thành $+1$;
- phép chia benchmark: kích thước, phân tầng, tập lồng nhau, không rò rỉ sang validation;
- **E1** không tạo `DataLoader` nào cho nhánh xoay; **E2-L** chỉ dùng D_L ; **E2** dùng đủ 45.000 ảnh của D_{train} ;
- dataset xoay không mang nhãn siêu lớp nên không thể rò rỉ nhãn;
- Macro-F1 khớp ví dụ tính tay và quy ước mẫu số bằng 0 cho kết quả 0;
- bộ phân loại luôn đoán `Animal` đạt đúng Accuracy 60 % trên phép chia này;
- cấu hình optimizer và scheduler đúng tham số, learning rate về 0 sau T_{max} bước;
- $\lambda_{\text{rot}} = 0$ **vẫn làm thay đổi thống kê BatchNorm** — kiểm chứng trực tiếp lập luận ở mục 7.3;
- khởi tạo theo seed là tái lập được;
- **nhánh contrastive**: NT-Xent khớp công thức tính tay và bằng $\log 3$ ở trường hợp suy biến; SupCon khớp công thức L_{out} tính tay và trả 0 (không NaN) khi không có positive pair — đây là hồi quy cho lỗi $-\infty \cdot 0$;
- Projection Head không làm thay đổi số tham số suy luận và không nhận gradient từ nhiệm vụ phân loại;
- dataset contrastive từ D_{train} **không** mang nhãn; hai view của cùng một ảnh thực sự khác nhau;
- ánh xạ **E3–E6** sang nhánh xoay và loại loss contrastive đúng bảng 5.

Kết quả lần chạy gần nhất: **49 bài kiểm thử ĐẠT, 0 thất bại** (`pytest tests -q`, log lưu ở `results/logs/pytest.log`). Ngoài ra `scripts/check_contrastive.py` chạy kiểm tra độc lập cho nhánh

contrastive và trả mã thoát khác 0 nếu bất kỳ kiểm tra nào không đạt.

A.5 Bảng số liệu dạng CSV

Danh sách đầy đủ các chỉ số của từng lượt chạy, ở dạng dễ xử lý bằng máy, được lưu tại `results/runs_table.csv`. Tập này chứa: tag lượt chạy, cấu hình, mức nhãn, lần lặp, epoch được chọn, Macro-F1 và Accuracy trên validation, Macro-F1 và Accuracy trên test, F1 và precision/recall của từng siêu lớp, thời gian huấn luyện, và số tham số suy luận/huấn luyện. Ma trận nhầm lẫn đầy đủ của từng lượt nằm trong trường `test.confusion` của tập `results/checkpoints/<tag>.json` tương ứng (không kèm trong kho mã nguồn — xem hộp “Lưu ý về gói bàn giao” ở trên).

A.6 Đường học của toàn bộ lịch sử huấn luyện

Mỗi tập `results/checkpoints/<tag>.json` chứa trường `history` với đầy đủ 100 bản ghi epoch, mỗi bản ghi gồm: loss huấn luyện của nhánh có giám sát, loss huấn luyện của nhánh phụ (bằng `null` với **E1**), tổng loss, loss phân loại trên validation, Macro-F1 validation, Accuracy validation, learning rate hiện hành và thời gian của epoch. Nhờ vậy, mọi hình trong báo cáo đều có thể được dựng lại từ dữ liệu thô mà không cần chạy lại mô hình.

A.7 Ghi nhận về các thiết lập được đề xuất

Một số hằng số trong báo cáo là *thiết lập được khóa trước để khép kín giao thức*, không phải giá trị do giảng viên hay sinh viên xác nhận trước khi chạy: seed chia split 2026, hằng số chuẩn hóa (0,5; 0,5; 0,5), quy tắc lấy minibatch cho nhánh phụ, và số lượt đo độ trễ (50 lượt khởi động + 1.000 lượt đo × ba phiên). Các giá trị này được chốt *trước* khi chạy để tránh việc điều chỉnh giao thức sau khi đã thấy kết quả test.

Tài liệu tham khảo

- [1] K. He, X. Zhang, S. Ren, and J. Sun, Deep residual learning for image recognition, in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, Las Vegas, NV, USA, 2016, pp. 770–778. DOI: [10.1109/CVPR.2016.90](https://doi.org/10.1109/CVPR.2016.90). Preprint: [arXiv:1512.03385](https://arxiv.org/abs/1512.03385).
- [2] S. Gidaris, P. Singh, and N. Komodakis, Unsupervised representation learning by predicting image rotations, in *International Conference on Learning Representations (ICLR)*, Vancouver, Canada, 2018. Preprint: [arXiv:1803.07728](https://arxiv.org/abs/1803.07728).
- [3] T. Chen, S. Kornblith, M. Norouzi, and G. Hinton, A simple framework for contrastive learning of visual representations, in *Proceedings of the 37th International Conference on Machine Learning (ICML)*, PMLR 119, 2020, pp. 1597–1607. Preprint: [arXiv:2002.05709](https://arxiv.org/abs/2002.05709).
- [4] P. Khosla, P. Teterwak, C. Wang, A. Sarna, Y. Tian, P. Isola, A. Maschinot, C. Liu, and D. Krishnan, Supervised contrastive learning, in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020, pp. 18661–18673. Preprint: [arXiv:2004.11362](https://arxiv.org/abs/2004.11362).
- [5] A. Krizhevsky, *Learning Multiple Layers of Features from Tiny Images*, Technical report, University of Toronto, 2009. Truy cập: <https://www.cs.toronto.edu/~kriz/learning-features-2009-TR.pdf>. Trang dữ liệu chính thức: <https://www.cs.toronto.edu/~kriz/cifar.html>.
- [6] PyTorch Contributors, *torchvision.models.resnet* — tài liệu và mã nguồn TorchVision, PyTorch 2.11 Documentation. Truy cập: <https://pytorch.org/vision/stable/models/resnet.html>.
- [7] PyTorch Contributors, *CrossEntropyLoss* — PyTorch documentation, PyTorch 2.11 Documentation. Truy cập: <https://pytorch.org/docs/stable/generated/torch.nn.CrossEntropyLoss.html>.
- [8] PyTorch Contributors, *PyTorch Benchmark* — hướng dẫn đo hiệu năng chính thức, PyTorch Tutorials. Truy cập: <https://pytorch.org/tutorials/recipes/recipes/benchmark.html>.
- [9] PyTorch Contributors, *Reproducibility* — PyTorch documentation, PyTorch 2.11 Documentation. Truy cập: <https://pytorch.org/docs/stable/notes/randomness.html>.
- [10] PyTorch Contributors, *torch.rot90* — PyTorch documentation, PyTorch 2.11 Documentation. Truy cập: <https://pytorch.org/docs/stable/generated/torch.rot90.html>.
- [11] I. Loshchilov and F. Hutter, Decoupled weight decay regularization, in *International Conference on Learning Representations (ICLR)*, New Orleans, LA, USA, 2019. Preprint: [arXiv:1711.05101](https://arxiv.org/abs/1711.05101).
- [12] I. Loshchilov and F. Hutter, SGDR: Stochastic gradient descent with warm restarts, in *International Conference on Learning Representations (ICLR)*, Toulon, France, 2017. Preprint: [arXiv:1608.03983](https://arxiv.org/abs/1608.03983).
- [13] S. Ioffe and C. Szegedy, Batch normalization: Accelerating deep network training by reducing internal covariate shift, in *Proceedings of the 32nd International Conference on Machine Learning (ICML)*, PMLR 37, 2015, pp. 448–456. Preprint: [arXiv:1502.03167](https://arxiv.org/abs/1502.03167).
- [14] R. Caruana, Multitask learning, *Machine Learning*, vol. 28, no. 1, pp. 41–75, 1997. DOI: [10.1023/A:1007379606734](https://doi.org/10.1023/A:1007379606734).
- [15] S. Ruder, An overview of multi-task learning in deep neural networks, *arXiv preprint*, 2017. [arXiv:1706.05098](https://arxiv.org/abs/1706.05098).
- [16] C. Doersch, A. Gupta, and A. A. Efros, Unsupervised visual representation learning by context prediction, in *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, Santiago, Chile, 2015, pp. 1422–1430. Preprint: [arXiv:1505.05192](https://arxiv.org/abs/1505.05192).
- [17] M. Noroozi and P. Favaro, Unsupervised learning of visual representations by solving jigsaw puzzles, in *European Conference on Computer Vision (ECCV)*, Amsterdam, The Netherlands, 2016, pp. 69–84. Preprint: [arXiv:1603.09246](https://arxiv.org/abs/1603.09246).

- [18] H. Zhang, M. Cissé, Y. N. Dauphin, and D. Lopez-Paz, mixup: Beyond empirical risk minimization, in *International Conference on Learning Representations (ICLR)*, Vancouver, Canada, 2018. Preprint: [arXiv:1710.09412](#).
- [19] C. Banbury *et al.*, MLPerf Tiny benchmark, in *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks*, 2021. Preprint: [arXiv:2106.07597](#).
- [20] J. Lin, W.-M. Chen, Y. Lin, J. Cohn, C. Gan, and S. Han, MCUNet: Tiny deep learning on IoT devices, in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020, pp. 11711–11722. Preprint: [arXiv:2007.10319](#).
- [21] A. G. Howard *et al.*, MobileNets: Efficient convolutional neural networks for mobile vision applications, *arXiv preprint*, 2017. [arXiv:1704.04861](#).
- [22] C. Szegedy, V. Vanhoucke, S. Ioffe, J. Shlens, and Z. Wojna, Rethinking the inception architecture for computer vision, in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, Las Vegas, NV, USA, 2016, pp. 2818–2826. Preprint: [arXiv:1512.00567](#).
- [23] C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger, On calibration of modern neural networks, in *Proceedings of the 34th International Conference on Machine Learning (ICML)*, PMLR 70, 2017, pp. 1321–1330. Preprint: [arXiv:1706.04599](#).
- [24] X. Bouthillier *et al.*, Accounting for variance in machine learning benchmarks, in *Proceedings of Machine Learning and Systems (MLSys)*, vol. 3, 2021, pp. 747–769. Preprint: [arXiv:2103.03098](#).
- [25] J. Oliver, P. Oden, and C. Shin, Realistic evaluation of deep semi-supervised learning algorithms, in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 31, 2018, pp. 3235–3246. Preprint: [arXiv:1804.09170](#).